

ROBOT MANIPOLATORE PLANARE 4R CON GRAVITÀ

Documentazione tecnica di progetto

Cinematica diretta e inversa · Dinamica di Eulero–Lagrange · Controllo di posizione · Simulazione e animazione in MATLAB

Voce	Contenuto
Ateneo	Università San Raffaele Roma
Studente & Matricola	Daniele Murgese (0322500069)
Ambiente	MATLAB R2019b o successivo (compatibile con GNU Octave 8+)
Toolbox	Nessuno per l'esecuzione; Symbolic Math Toolbox solo per la derivazione simbolica
Punto di ingresso	<code>main_4R.m</code> (esecuzione completa), <code>test_4R.m</code> (verifiche numeriche)
Parametri	<code>params_4R.m</code> – tutti i valori numerici sono configurabili dall'utente
Versione documento	1.0 – ottobre 2026
Link Repository	https://github.com/danielemurgese992/Robotica-intelligente

Indice

1. Introduzione e obiettivi	5
1.1 Corrispondenza tra consegne e implementazione	5
2. Struttura del progetto e avvio	6
2.1 Requisiti software	6
2.2 File del progetto	6
2.3 Avvio rapido	6
3. Parametri configurabili	8
3.1 Robot (p.robot)	8
3.2 Cinematica inversa (p.ik)	8
3.3 Traiettorie (p.traj)	8
3.4 Controllore (p.ctrl)	9
3.5 Simulazione, visualizzazione ed esecuzione	9
4. Cinematica diretta	10
4.1 Assegnazione delle terne secondo Denavit–Hartenberg	10
4.2 Tabella DH	10
4.3 Matrici di trasformazione A_i	11
4.4 Trasformazione complessiva	11
4.5 Posizione e orientamento dell'end-effector	11
4.6 Implementazione MATLAB	12
4.7 Grafico del robot in una configurazione	12
5. Cinematica inversa	13
5.1 Equazione del polso	13
5.2 Ridondanza e parametro ψ	13
5.3 Gruppo 3R e problema 2R	13
5.4 Quarto giunto	13
5.5 Soluzioni multiple: gomito alto e gomito basso	13
5.6 Implementazione e verifica FK(IK(p))	14
6. Jacobiano	15
7. Dinamica con il metodo di Eulero–Lagrange	16
7.1 Ipotesi e coordinate	16
7.2 Posizione e velocità dei baricentri	16
7.3 Energia cinetica e matrice di inerzia	16
7.4 Energia potenziale e vettore gravitazionale	16
7.5 Equazioni di Eulero–Lagrange	16
7.6 Derivazione simbolica in MATLAB	17
7.7 Funzione richiesta	17

8. Traiettoria di riferimento	18
8.1 Legge oraria	18
8.2 Percorsi disponibili	18
8.3 Conversione nello spazio dei giunti	18
9. Progetto del controllore	19
9.1 Scelta: controllo a coppia calcolata	19
9.2 Alternativa: PD con compensazione di gravità	19
9.3 Robustezza e saturazione	19
10. Simulazione con ode45	20
11. Grafici, visualizzazione e animazione	21
11.1 Grafici richiesti: $q(t)$, $e(t)$, $\tau(t)$	21
11.2 Animazione	22
12. Test e verifica dei risultati	24
12.1 Risultati della simulazione in diversi scenari	24
13. Guida passo-passo per lanciare e testare il progetto	26
13.1 Prima esecuzione	26
13.2 Scegliere i valori di prova	26
13.3 Usare le funzioni singolarmente	26
13.4 Risoluzione dei problemi	27
13.5 Esecuzione in GNU Octave	27
Appendice A – Codice sorgente completo	28
A.1 params_4R.m	28
A.2 main_4R.m	29
A.3 dh_table_4R.m	30
A.4 dh_matrix.m	30
A.5 fk_4R.m	31
A.6 ik_4R.m	31
A.7 jacobian_4R.m	33
A.8 dynamics_4R.m	33
A.9 energy_4R.m	34
A.10 derive_dynamics_4R_sym.m	34
A.11 task_reference_4R.m	36
A.12 build_reference_4R.m	37
A.13 reference_4R.m	37
A.14 controller_4R.m	37
A.15 robot_ode_4R.m	38
A.16 simulate_4R.m	38
A.17 plot_robot_4R.m	39

A.18 plot_results_4R.m	40
A.19 animate_4R.m	40
A.20 test_4R.m	41

1. Introduzione e obiettivi

Il progetto riguarda un **robot manipolatore planare a 4 gradi di libertà** composto da quattro giunti rotoidali (4R), con assi di rotazione paralleli tra loro e ortogonali al piano di lavoro XY. I link si muovono nel piano XY, i giunti ruotano attorno agli assi z_i e la gravità agisce lungo l'asse $-y$.

Gli obiettivi richiesti dalla traccia sono quattro:

1. modellare completamente il robot dal punto di vista cinematico e dinamico;
2. implementare tali modelli in MATLAB;
3. progettare e simulare un controllore di posizione lungo una traiettoria di riferimento;
4. verificare la correttezza dei risultati tramite animazioni e test numerici.

Questo documento descrive la teoria utilizzata, le scelte progettuali, la struttura del codice, la procedura per configurare e lanciare il progetto e i risultati ottenuti. In Appendice A è riportato il codice sorgente completo.

1.1 Corrispondenza tra consegne e implementazione

Consegna	Dove è implementata	Sezione
3.1 Cinematica diretta (DH, A_i , T, x, y, φ , grafico)	dh_table_4R, dh_matrix, fk_4R, plot_robot_4R	4
3.2 Cinematica inversa (polso, 3R + θ_4 , gomito alto/basso, verifica)	ik_4R, test_4R, main_4R (sezione 2)	5
3.3 Dinamica con Eulero–Lagrange (T, V, M, C, G, simbolico)	dynamics_4R, derive_dynamics_4R_sym, energy_4R	7
4 Progetto del controllore + traiettoria + ode45	controller_4R, task_reference_4R, reference_4R, simulate_4R	8, 9, 10
4.1 Grafici $q(t)$, $e(t)$, $\tau(t)$	plot_results_4R	11
5 Animazione con terne DH e target (drawnow)	animate_4R, plot_robot_4R	11
Verifica numerica	test_4R	12

2. Struttura del progetto e avvio

2.1 Requisiti software

- **MATLAB R2019b o successivo:** nessun toolbox è necessario per eseguire simulazione, grafici, animazione e test.
- **Symbolic Math Toolbox** (opzionale): serve solo per `derive_dynamics_4R_sym.m`, la derivazione simbolica di M , C e G .
- **GNU Octave 8+** (alternativa gratuita): il codice è compatibile ed è stato eseguito con Octave 8.4. Per la parte simbolica serve il pacchetto `symbolic (pkg install -forge symbolic)`.

2.2 File del progetto

File	Tipo	Ruolo
main_4R.m	script	Punto di ingresso: esegue FK, IK, dinamica, simulazione, grafici e animazione
params_4R.m	funzione	Restituisce la struttura p con tutti i parametri configurabili
dh_table_4R.m	funzione	Tabella DH $[a \ d \ \theta]$ del 4R
dh_matrix.m	funzione	Matrice omogenea A_i (convenzione DH standard)
fk_4R.m	funzione	$[T, x, y, \phi] = \text{fk_4R}(\theta, L)$ – cinematica diretta
ik_4R.m	funzione	$\theta = \text{ik_4R}(x, y, \phi, L)$ – cinematica inversa analitica
jacobian_4R.m	funzione	Jacobiano planare $J(q)$ e sua derivata temporale
dynamics_4R.m	funzione	$[M, C, G] = \text{dynamics_4R}(\theta, d\theta, \text{params})$ – modello dinamico
energy_4R.m	funzione	Energia cinetica e potenziale (per le verifiche)
derive_dynamics_4R_sym.m	funzione	Derivazione simbolica di Eulero–Lagrange e confronto
task_reference_4R.m	funzione	Traiettoria cartesiana (cerchio/retta) con derivate analitiche
build_reference_4R.m	funzione	Pre-calcolo e validazione della traiettoria (raggiungibilità, singolarità)
reference_4R.m	funzione	Riferimento di giunto $q_d, \dot{q}_d, \ddot{q}_d$ all'istante t
controller_4R.m	funzione	Coppia calcolata oppure PD + compensazione di gravità
robot_ode_4R.m	funzione	Equazioni di stato in anello chiuso per ode45
simulate_4R.m	funzione	Lancia ode45 e raccoglie i risultati
plot_robot_4R.m	funzione	Disegno del robot e delle terne DH
plot_results_4R.m	funzione	Grafici $q(t)$, $e(t)$, $\tau(t)$ e percorso cartesiano
animate_4R.m	funzione	Animazione con <code>drawnow</code> (opzionale salvataggio video)
test_4R.m	funzione	Batteria di test numerici con esito PASS/FAIL
README.md	testo	Istruzioni rapide

2.3 Avvio rapido

1. Estrarre l'archivio `Robot4R.zip` in una cartella a scelta.

2. Aprire MATLAB e impostare quella cartella come **Current Folder** (oppure eseguire `cd percorso/Robot4R`).
3. (Facoltativo) Modificare i valori in `params_4R.m` o nella sezione PERSONALIZZAZIONE di `main_4R.m`.
4. Digitare `main_4R` nella Command Window e premere Invio.
5. Digitare `test_4R` per eseguire le verifiche numeriche.
6. (Facoltativo, con Symbolic Math Toolbox) eseguire `S = derive_dynamics_4R_sym(params_4R(), true);`.

La procedura dettagliata, con gli scenari di prova e la risoluzione dei problemi più comuni, è descritta nella Sezione 13.

3. Parametri configurabili

Nessun valore numerico è scritto nelle funzioni: tutti si trovano in `params_4R.m`, che restituisce una struttura `p` organizzata in sotto-strutture. Ci sono due modi per cambiarli:

- modificare direttamente `params_4R.m` (le modifiche valgono per tutti gli script);
- sovrascrivere i campi in `main_4R.m`, nella sezione PERSONALIZZAZIONE, dopo `p = params_4R();` (utile per esperimenti rapidi).

Unità di misura del Sistema Internazionale in tutto il progetto: m, kg, $\text{kg}\cdot\text{m}^2$, s, rad, N·m. Per gli angoli si può usare `deg2rad(...)`.

3.1 Robot (p.robot)

Campo	Default	Unità	Significato
L	[0.40 0.35 0.25 0.15]	m	Lunghezze dei link L1...L4
m	[3.0 2.5 1.5 0.8]	kg	Masse dei link m1...m4 (concentrate nei baricentri)
lc	L/2	m	Distanza del baricentro dal giunto a monte
I	$m\cdot L^2/12$	$\text{kg}\cdot\text{m}^2$	Momenti di inerzia baricentrici (asta sottile)
g	9.81	m/s^2	Accelerazione di gravità, diretta lungo $-y$

3.2 Cinematica inversa (p.ik)

Campo	Default	Unità	Significato
elbow	'up'	–	Configurazione del gomito: 'up' (alto) o 'down' (basso)
psiOffset	<code>deg2rad(-40)</code>	rad	Ridondanza: $\psi(t) = \phi(t) + \text{psiOffset}$, con $\psi = \theta_1 + \theta_2 + \theta_3$ (Sezione 5.2)

3.3 Traiettoria (p.traj)

Campo	Default	Unità	Significato
type	'circle'	–	'circle' (cerchio) o 'line' (segmento)
T	6	s	Durata del moto (legge oraria quintica)
tHold	1	s	Permanenza a riposo nella posa finale
center, radius	[0.45 0.35], 0.15	m	Centro e raggio del cerchio
theta0, nTurns	0, 1	rad, –	Angolo polare iniziale e numero di giri
pStart, pEnd	[0.60 -0.10], [0.25 0.55]	m	Estremi del segmento
phiStart, phiEnd	0°, 30°	rad	Orientamento dell'end-effector, interpolato lungo il moto

3.4 Controllore (p.ctrl)

Campo	Default	Unità	Significato
type	'computed_torque'	–	'computed_torque' o 'pd_gravity'
Kp, Kd	$\omega_n^2 I, 2\zeta\omega_n I$ ($\omega_n = 10, \zeta = 1$)	1/s ² , 1/s	Guadagni della coppia calcolata
Ki	zeros(4)	1/s ³	Azione integrale della coppia calcolata (0 = disattivata)
pd.Kp, pd.Kd	diag([300 200 60 10]), diag([40 25 6 0.5])	N·m/rad, N·m·s/rad	Guadagni del PD + gravità
pd.Ki	zeros(4)	N·m/(rad·s)	Azione integrale del PD
modelScale	1.0	–	Fattore su masse e inerzie del modello usato dal controllore (test di robustezza)
tauMax	Inf	N·m	Saturazione delle coppie (scalare o vettore 1×4)

3.5 Simulazione, visualizzazione ed esecuzione

Campo	Default	Significato
sim.q0Offset	deg2rad([5 -5 5 -5])	Errore iniziale aggiunto a $q_d(0)$, per osservare la convergenza
sim.dq0	[0 0 0 0]	Velocità iniziali [rad/s]
sim.dtOut	0.01	Passo di campionamento dei risultati [s]
sim.RelTol, sim.AbsTol	1e-6, 1e-8	Tolleranze di ode45
viz.frameScale	0.10	Lunghezza degli assi delle terne DH disegnate [m]
viz.animStep, viz.pause	3, 0	Un fotogramma ogni animStep campioni; pausa tra fotogrammi [s]
viz.saveVideo, viz.videoFile	false, robot4R_animazione.mp4	Salvataggio dell'animazione (solo MATLAB)
viz.qDemo	deg2rad([30 45 -30 20])	Configurazione usata per i grafici di FK/IK
run.fkDemo ... run.animate	true	Attivano o disattivano le sezioni di main_4R

4. Cinematica diretta

4.1 Assegnazione delle terne secondo Denavit–Hartenberg

Si usa la convenzione DH **standard** (distale): la terna i è solidale al link i e ha origine all'estremità del link. Le regole applicate al 4R planare sono:

- **asse z_i** : coincide con l'asse del giunto $i+1$. Poiché tutti i giunti sono rotoidali con assi paralleli e ortogonali al piano, tutti gli z_i sono paralleli e uscenti dal piano XY;
- **terna base 0**: origine nel giunto 1, x_0 orizzontale, y_0 verticale verso l'alto (la gravità è quindi lungo $-y_0$);
- **asse x_i** : diretto lungo la normale comune tra z_{i-1} e z_i . Con assi paralleli la normale comune è la retta che congiunge i due giunti, cioè il link i . x_i punta quindi dal giunto i al giunto $i+1$;
- **asse y_i** : completa la terna destrorsa ($y_i = z_i \times x_i$);
- **terna 4**: posta all'estremità del link 4 e coincidente con la terna dell'end-effector, con x_4 lungo il link 4.

Con queste scelte, per ogni giunto: la distanza tra gli assi z_{i-1} e z_i lungo x_i è $a_i = L_i$; l'angolo di twist tra z_{i-1} e z_i è $\alpha_i = 0$ (assi paralleli); lo spostamento lungo z_{i-1} è $d_i = 0$ (tutti i link nello stesso piano); l'angolo θ_i tra x_{i-1} e x_i attorno a z_{i-1} è la variabile di giunto.

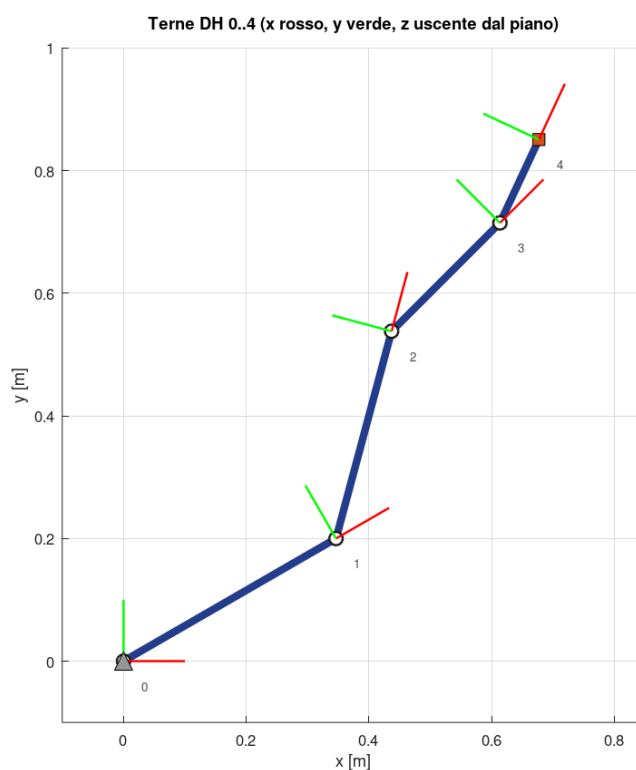


Figura 1 – Terne DH 0...4 nella configurazione di prova (x in rosso, y in verde, z uscente dal piano).

4.2 Tabella DH

Giunto i	a_i [m]	α_i [rad]	d_i [m]	θ_i [rad]
1	L_1	0	0	θ_1 (variabile)
2	L_2	0	0	θ_2 (variabile)

Giunto i	a_i [m]	α_i [rad]	d_i [m]	θ_i [rad]
3	L3	0	0	θ_3 (variabile)
4	L4	0	0	θ_4 (variabile)

4.3 Matrici di trasformazione A_i

La matrice DH standard è il prodotto di quattro trasformazioni elementari:

$$A_i = \text{Rot}_z(\theta_i) \cdot \text{Trans}_z(d_i) \cdot \text{Trans}_x(a_i) \cdot \text{Rot}_x(\alpha_i) \quad (1)$$

In forma generale:

$$A_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sostituendo $\alpha_i = 0$, $d_i = 0$ e $a_i = L_i$, con la notazione compatta $c_i = \cos \theta_i$ e $s_i = \sin \theta_i$:

$$A_i = \begin{bmatrix} c_i & -s_i & 0 & L_i c_i \\ s_i & c_i & 0 & L_i s_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Ogni A_i è quindi una rotazione di θ_i attorno a z seguita da una traslazione di L_i lungo il nuovo asse x. La funzione `dh_matrix(a, alpha, d, theta)` implementa la forma generale (1); `dh_table_4R(theta, L)` restituisce la tabella della Sezione 4.2.

4.4 Trasformazione complessiva

$$T = A_1 A_2 A_3 A_4 \quad (2)$$

Poiché le rotazioni avvengono tutte attorno ad assi z paralleli, gli angoli si sommano. Usando la notazione $c_{12} = \cos(\theta_1 + \theta_2)$, $c_{123} = \cos(\theta_1 + \theta_2 + \theta_3)$, $c_{1234} = \cos(\theta_1 + \theta_2 + \theta_3 + \theta_4)$ (e analoghe per il seno) si ottiene:

$$T = \begin{bmatrix} c_{1234} & -s_{1234} & 0 & x \\ s_{1234} & c_{1234} & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.5 Posizione e orientamento dell'end-effector

Dall'ultima colonna e dal blocco di rotazione di T si estraggono esplicitamente:

$$x = L_1 c_1 + L_2 c_{12} + L_3 c_{123} + L_4 c_{1234} \quad (3)$$

$$y = L_1 s_1 + L_2 s_{12} + L_3 s_{123} + L_4 s_{1234} \quad (4)$$

$$\varphi = \theta_1 + \theta_2 + \theta_3 + \theta_4 = \text{atan2}(T_{21}, T_{11}) \quad (5)$$

Nel codice φ è calcolato con `atan2(T(2,1), T(1,1))`, quindi è sempre ricondotto all'intervallo $(-\pi, \pi]$: il valore coincide con $\theta_1 + \theta_2 + \theta_3 + \theta_4$ a meno di multipli di 2π .

4.6 Implementazione MATLAB

`[T, x, y, phi, Tall] = fk_4R(theta, L)` costruisce la tabella DH, moltiplica in sequenza le matrici A_i e salva tutte le terne intermedie T_0^i nell'array `Tall` ($4 \times 4 \times 5$). Queste terne servono per disegnare il robot e le terne DH. La firma richiesta dalla traccia, `[T, x, y, phi] = fk_4R(theta, L)`, è rispettata: `Tall` è un'uscita aggiuntiva facoltativa.

4.7 Grafico del robot in una configurazione

La sezione 1 di `main_4R` stampa tabella DH, matrici A_i e T per `p.viz.qDemo = [30°, 45°, -30°, 20°]` e disegna il robot con `plot_robot_4R`. Con i parametri di default si ottiene $x = 0.6772$ m, $y = 0.8508$ m e $\varphi = 65^\circ$, che coincide con la somma degli angoli di giunto.

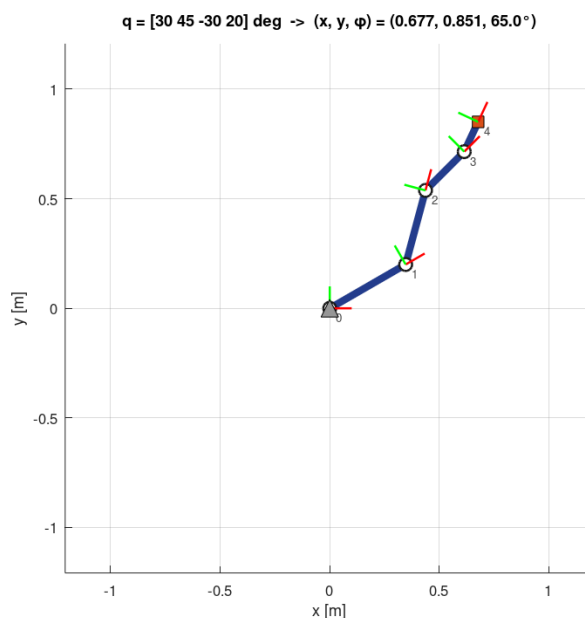


Figura 2 – Grafico del robot nella configurazione di prova (`main_4R`, sezione 1).

5. Cinematica inversa

5.1 Equazione del polso

Il "polso" è il punto W all'inizio del link 4, cioè l'origine della terna 3. Il link 4 ha lunghezza nota L_4 ed è orientato esattamente come l'end-effector (angolo assoluto φ), quindi W si ottiene arretrando di L_4 lungo la direzione φ :

$$x_w = x - L_4 \cos \varphi, \quad y_w = y - L_4 \sin \varphi \quad (6)$$

5.2 Ridondanza e parametro ψ

Il compito (x, y, φ) ha 3 componenti, mentre il robot ha 4 giunti: il 4R planare è **cinematicamente ridondante** di grado 1 e ammette infinite soluzioni (∞^1). La traccia suggerisce di risolvere $\theta_1, \theta_2, \theta_3$ come un problema 3R planare e di ricavare θ_4 per differenza. Un 3R planare è determinato quando sono assegnati sia la posizione del suo estremo (il polso W) sia l'orientamento del suo ultimo link. Si introduce quindi il **parametro di ridondanza**:

$$\psi = \theta_1 + \theta_2 + \theta_3 \quad (\text{orientamento assoluto del link 3}) \quad (7)$$

ψ è scelto dall'utente. In `ik_4R` è il sesto argomento facoltativo. Durante la traiettoria si pone $\psi(t) = \varphi(t) + \text{p.ik.psiOffset}$. Se ψ non è specificato, `ik_4R` prova $\psi = \varphi$ ($\theta_4 = 0$); se tale scelta non è raggiungibile, cerca sulla griglia $[\varphi - \pi, \varphi + \pi]$ il valore raggiungibile più vicino a φ .

5.3 Gruppo 3R e problema 2R

Noti W e ψ , il punto P2 all'estremità del link 2 si ottiene arretrando di L_3 lungo ψ :

$$x_2 = x_w - L_3 \cos \psi, \quad y_2 = y_w - L_3 \sin \psi \quad (8)$$

Il problema si riduce a un 2R planare con lunghezze L_1 e L_2 che deve raggiungere P2. Applicando il teorema del coseno al triangolo formato da base, gomito e P2:

$$c_2 = (x_2^2 + y_2^2 - L_1^2 - L_2^2) / (2 L_1 L_2) \quad (9)$$

$$s_2 = \mp \sqrt{1 - c_2^2} \quad \theta_2 = \text{atan2}(s_2, c_2) \quad (10)$$

$$\theta_1 = \text{atan2}(y_2, x_2) - \text{atan2}(L_2 s_2, L_1 + L_2 c_2) \quad (11)$$

$$\theta_3 = \psi - \theta_1 - \theta_2 \quad (12)$$

5.4 Quarto giunto

$$\theta_4 = \varphi - (\theta_1 + \theta_2 + \theta_3) = \varphi - \psi \quad (13)$$

5.5 Soluzioni multiple: gomito alto e gomito basso

Per ogni valore di ψ , l'equazione (10) fornisce due soluzioni distinte, legate al segno di s_2 :

- **gomito alto** (`elbow = 'up'`): $s_2 < 0$, $\theta_2 < 0$. Il gomito si trova a sinistra della congiungente base-P2, cioè "sopra" per bersagli nel primo quadrante;
- **gomito basso** (`elbow = 'down'`): $s_2 > 0$, $\theta_2 > 0$. Il gomito si trova "sotto".

Le due soluzioni hanno lo stesso θ_4 e lo stesso ψ , ma valori diversi di θ_1 , θ_2 e θ_3 . `ik_4R` restituisce la soluzione selezionata in `theta` ed entrambe in `info.solutions` (4x2). Per la ridondanza, al variare di ψ esistono in realtà infinite coppie di soluzioni.

Condizioni di esistenza e singolarità:

- **raggiungibilità:** esiste soluzione solo se $|L1 - L2| \leq \sqrt{x^2 + y^2} \leq L1 + L2$, cioè se $|c2| \leq 1$. In caso contrario `ik_4R` restituisce NaN con `info.reachable = false`;
- **singolarità:** per $s2 = 0$ (braccio 2R completamente steso o ripiegato) le due soluzioni coincidono e il Jacobiano perde rango. `build_reference_4R` segnala con un warning le traiettorie che passano vicino a questa condizione ($|\sin \theta_2| < 0.05$).

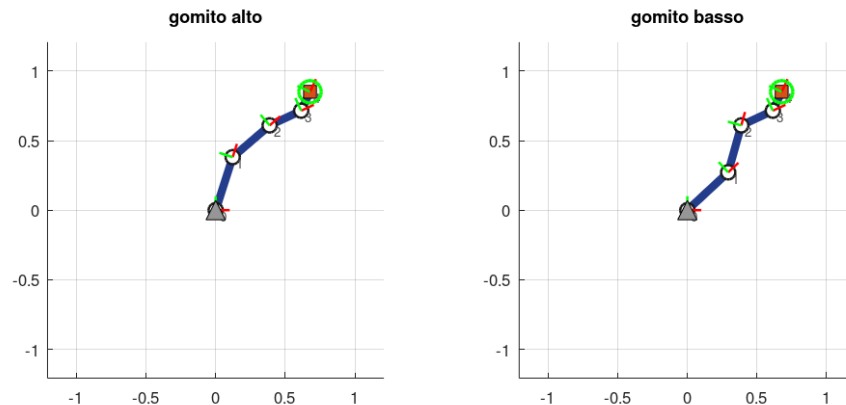


Figura 3 – Le due soluzioni (gomito alto e basso) per lo stesso target (x, y, ϕ) e lo stesso ψ .

Per il target di prova $(0.6772, 0.8508, 65^\circ)$ con $\psi = 25^\circ$ si ottengono $\theta = [72.27^\circ, -31.58^\circ, -15.69^\circ, 40.00^\circ]$ (gomito alto) e $\theta = [42.85^\circ, 31.58^\circ, -49.43^\circ, 40.00^\circ]$ (gomito basso). Per entrambe l'errore $\|FK(IK(p)) - p\|$ è dell'ordine di $1e-16$.

5.6 Implementazione e verifica $FK(IK(p))$

Firma richiesta: `theta = ik_4R(x, y, phi, L)`. Firma estesa: `[theta, info] = ik_4R(x, y, phi, L, elbow, psi)`. Gli angoli restituiti sono ricondotti in $(-\pi, \pi]$.

La verifica avviene in due punti. La sezione 2 di `main_4R` stampa l'errore $FK(IK(p)) - p$ per le due soluzioni. `test_4R` genera 200 configurazioni casuali q , calcola $p = FK(q)$, risolve IK con $\psi = \theta_1 + \theta_2 + \theta_3$ per entrambi i gomiti e controlla che $FK(IK(p)) = p$ e che q coincida con una delle due soluzioni. Ripete poi il controllo con ψ automatico e verifica che un punto fuori dallo spazio di lavoro venga riconosciuto come non raggiungibile.

6. Jacobiano

Il Jacobiano non è esplicitamente richiesto, ma serve per calcolare velocità e accelerazioni di riferimento ai giunti (Sezione 9). Detti $\varphi_k = \theta_1 + \dots + \theta_k$ gli angoli assoluti dei link, la velocità del compito vale $[\dot{x}, \dot{y}, \dot{\varphi}]^T = J(q) \dot{q}$ con:

$$J_{1j} = -\sum_{k \geq j} L_k \sin \varphi_k, \quad J_{2j} = \sum_{k \geq j} L_k \cos \varphi_k, \quad J_{3j} = 1 \quad (14)$$

$$J_{1j} = -\sum_{k \geq j} L_k \cos \varphi_k \dot{\varphi}_k, \quad J_{2j} = -\sum_{k \geq j} L_k \sin \varphi_k \dot{\varphi}_k, \quad J_{3j} = 0 \quad (15)$$

`[J, Jdot] = jacobian_4R(theta, L, dtheta)` implementa le (14)–(15). `test_4R` le confronta con le differenze finite centrate della cinematica diretta.

7. Dinamica con il metodo di Eulero–Lagrange

7.1 Ipotesi e coordinate

Ogni link i è un corpo rigido con massa m_i concentrata nel baricentro, posto a distanza l_{ci} dal giunto a monte lungo l'asse del link, e momento di inerzia baricentrico I_i attorno all'asse z . Le coordinate generalizzate sono $q = [\theta_1, \theta_2, \theta_3, \theta_4]^T$. Conviene usare anche gli angoli assoluti $\varphi = S q$, dove S è la matrice 4×4 triangolare inferiore di uni ($\varphi_k = \theta_1 + \dots + \theta_k$).

7.2 Posizione e velocità dei baricentri

Definendo i "bracci" $a_{ik} = L_k$ per $k < i$, $a_{ii} = l_{ci}$ e $a_{ik} = 0$ per $k > i$:

$$p_{ci} = \sum_{k \leq i} a_{ik} [\cos \varphi_k, \sin \varphi_k]^T \quad (16)$$

$$v_{ci} = \sum_{k \leq i} a_{ik} \dot{\varphi}_k [-\sin \varphi_k, \cos \varphi_k]^T, \quad \omega_i = \dot{\varphi}_i = \dot{\theta}_1 + \dots + \dot{\theta}_i \quad (17)$$

7.3 Energia cinetica e matrice di inerzia

$$T = \sum_{i=1}^4 \left(\frac{1}{2} m_i \|v_{ci}\|^2 + \frac{1}{2} I_i \omega_i^2 \right) = \frac{1}{2} \dot{q}^T M(q) \dot{q} \quad (18)$$

Nella forma Jacobiana classica $M(q) = \sum_i (m_i J_{vi}^T J_{vi} + I_i J_{\omega i}^T J_{\omega i})$, dove J_{vi} è il Jacobiano di posizione del baricentro i e $J_{\omega i} = [1 \dots 1 \ 0 \dots 0]$ (uni sulle prime i colonne). Sviluppando $\|v_{ci}\|^2$ con la (17) si ottiene la forma chiusa usata in `dynamics_4R`:

$$B_{kl} = \sum_{i \geq \max(k,l)} m_i a_{ik} a_{il} \quad (19)$$

$$M^{\varphi}_{kl} = B_{kl} \cos(\varphi_k - \varphi_l) + \delta_{kl} I_k, \quad M(q) = S^T M^{\varphi} S \quad (20)$$

M^{φ} è la matrice di inerzia espressa negli angoli assoluti; la trasformazione congruente $S^T(\cdot)S$ la riporta negli angoli relativi di giunto. $M(q)$ è simmetrica e definita positiva.

7.4 Energia potenziale e vettore gravitazionale

Con la gravità diretta lungo $-y$, la quota del baricentro i è $y_{ci} = \sum_{k \leq i} a_{ik} \sin \varphi_k$ e:

$$V = \sum_{i=1}^4 m_i g h_i(q) = \sum_{i=1}^4 m_i g y_{ci} \quad (21)$$

$$G^{\varphi}_k = \partial V / \partial \varphi_k = g \cos \varphi_k \sum_{i \geq k} m_i a_{ik}, \quad G(q) = \partial V / \partial q = S^T G^{\varphi} \quad (22)$$

7.5 Equazioni di Eulero–Lagrange

Con la lagrangiana $\mathcal{L} = T - V$, le equazioni del moto sono:

$$d/dt (\partial \mathcal{L} / \partial \dot{q}_k) - \partial \mathcal{L} / \partial q_k = \tau_k, \quad k = 1 \dots 4 \quad (23)$$

Sviluppando le derivate si ottiene la forma matriciale standard:

$$M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + G(q) = \tau \quad (24)$$

La matrice dei termini di Coriolis e centrifughi è costruita con i **simboli di Christoffel** di prima specie:

$$C_{kj} = \sum_{i=1}^4 \frac{1}{2} (\partial M_{kj} / \partial q_i + \partial M_{ki} / \partial q_j - \partial M_{ij} / \partial q_k) \dot{q}_i \quad (25)$$

Applicando la (25) a M^{φ} in coordinate assolute, i termini con δ si elidono e si ottiene una forma chiusa compatta. Poiché $\varphi = S q$ è una trasformazione lineare, i simboli di Christoffel si trasformano in modo tensoriale:

$$C^{\varphi}_{kl} = B_{kl} \sin(\varphi_k - \varphi_l) \dot{\varphi}_l, \quad C(q, \dot{q}) = S^T C^{\varphi} S \quad (26)$$

Con questa scelta di C vale la proprietà di **antisimmetria**: $\dot{M} - 2C$ è antisimmetrica. In coordinate assolute $\dot{M}^\phi - 2C^\phi = -W \circ (\dot{\phi}_k + \dot{\phi}_l)$, con $W_{kl} = B_{kl} \sin(\phi_k - \phi_l)$ antisimmetrica. La proprietà è utile per le dimostrazioni di stabilità dei controllori basati su Lyapunov.

7.6 Derivazione simbolica in MATLAB

La traccia chiede di ricavare M , G e C in MATLAB simbolico. `derive_dynamics_4R_sym(p, doGenerate)` esegue la derivazione completa partendo dalle matrici DH simboliche:

1. costruisce le terne con `dh_matrix` su variabili simboliche e ricava i baricentri;
2. calcola $T = \sum \frac{1}{2} m_i v_{ci}^T v_{ci} + \frac{1}{2} I_i \omega_i^2$ e $V = \sum m_i g y_{ci}$;
3. ottiene $M_{kj} = \partial^2 T / \partial \dot{q}_k \partial \dot{q}_j$ e $G_k = \partial V / \partial q_k$;
4. costruisce C con i simboli di Christoffel (25), in forma matriciale: detta $D_i = \partial M / \partial q_i$ e $V = [D_1 q, \dots, D_4 q]$, vale $C = \frac{1}{2}(M + V - V^T)$;
5. verifica che le equazioni di Lagrange (23), calcolate direttamente come derivata totale di $\partial \mathcal{L} / \partial \dot{q}$, coincidano con $M \ddot{q} + C \dot{q} + G$;
6. confronta numericamente il risultato con `dynamics_4R` in configurazioni casuali;
7. se `doGenerate = true` (solo MATLAB), genera con `matlabFunction` i file `M_4R_sym.m`, `C_4R_sym.m` e `G_4R_sym.m` nella cartella `generated`.

Perché due implementazioni? `dynamics_4R` usa le forme chiuse (20), (22) e (26): è veloce e non richiede toolbox, quindi può essere chiamata migliaia di volte da `ode45`. La derivazione simbolica è il riferimento teorico che ne certifica la correttezza. Il confronto tra le due è stato eseguito anche con un'implementazione SymPy indipendente: la differenza massima su M , C e G è risultata $7 \cdot 10^{-15}$.

Si consiglia di eseguire la derivazione simbolica in MATLAB. In GNU Octave il pacchetto symbolic passa ogni operazione a Python e le espressioni non semplificate di un 4R diventano molto grandi: la derivazione richiede tempi molto lunghi e diversi GB di memoria. Per questo in Octave la semplificazione è disattivata automaticamente. Tutte le altre funzioni del progetto funzionano in Octave senza limitazioni.

7.7 Funzione richiesta

`[M, C, G] = dynamics_4R(theta, dtheta, params)` accetta come `params` sia `p.robot` sia l'intera struttura `p`. Esempio di valori con i parametri di default, $q = q_{\text{Demo}}$ e $\dot{q} = 0.5$ rad/s per ogni giunto: $M_{11} = 2.745 \text{ kg} \cdot \text{m}^2$, $G = [27.50, 6.09, 2.94, 0.25]^T \text{ N} \cdot \text{m}$.

8. Traiettoria di riferimento

8.1 Legge oraria

Per evitare discontinuità di velocità e accelerazione, che produrrebbero picchi di coppia, l'ascissa curvilinea normalizzata $s \in [0, 1]$ segue un polinomio quintico con velocità e accelerazione nulle agli estremi:

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \tau = t / T \quad (27)$$

Per $t > T$ il riferimento resta fermo nella posa finale per `tHold` secondi, così da osservare la regolazione a regime.

8.2 Percorsi disponibili

- **Cerchio** (`type = 'circle'`): $p(t) = c + r [\cos \vartheta, \sin \vartheta]^T$ con $\vartheta = \vartheta_0 + 2\pi \cdot nTurns \cdot s(t)$.
- **Segmento** (`type = 'line'`): $p(t) = p_start + (p_end - p_start) s(t)$.

L'orientamento è interpolato con la stessa legge, $\varphi(t) = \varphi_start + (\varphi_end - \varphi_start) s(t)$, e il parametro di ridondanza vale $\psi(t) = \varphi(t) + \psiOffset$. Tutte le derivate prime e seconde sono calcolate analiticamente in `task_reference_4R`.

8.3 Conversione nello spazio dei giunti

Il controllore lavora nello spazio dei giunti e richiede q_d , $\dot{q}_d$ e $\ddot{q}_d$. Si definisce il **Jacobiano aumentato** 4×4 , che lega $\dot{q}$ a $[\dot{x}, \dot{y}, \dot{\varphi}, \dot{\psi}]^T$ e rende quadrato il problema ridondante:

$$J_a = [J_{1,:}; J_{2,:}; 1 \ 1 \ 1 \ 1; 1 \ 1 \ 1 \ 0] \quad (28)$$

$$q_d = IK(x_d, y_d, \varphi_d, \psi_d), \quad \dot{q}_d = J_a^{-1} \dot{X}_d, \quad \ddot{q}_d = J_a^{-1} (\ddot{X}_d - \dot{J}_a \dot{q}_d) \quad (29)$$

`build_reference_4R` campiona l'intera traiettoria su una griglia fine e svolge tre compiti: verifica che ogni punto sia raggiungibile (altrimenti si ferma con un messaggio che indica istante e posa critici), segnala la vicinanza a singolarità e applica `unwrap` agli angoli per ottenere un riferimento continuo. Durante la simulazione `reference_4R` ricalcola q_d con la IK analitica e la riporta sullo stesso ramo continuo (multipli di 2π).

9. Progetto del controllore

9.1 Scelta: controllo a coppia calcolata

Il controllore scelto è la **coppia calcolata** (computed torque, o inverse dynamics control), nello spazio dei giunti. Gli errori sono $e = q_d - \dot{q}$ ed $\dot{e} = \ddot{q}_d - \ddot{q}$; la legge di controllo è:

$$\tau = \hat{M}(q) (\ddot{q}_d + K_d \dot{e} + K_p e + K_i \int e dt) + \hat{C}(q, \dot{q}) \dot{q} + \hat{G}(q) \quad (30)$$

Il controllore usa il modello dinamico ($\hat{M}$, $\hat{C}$, $\hat{G}$) per cancellare le non linearità del robot. Con modello esatto ($\hat{M} = M$, $\hat{C} = C$, $\hat{G} = G$), sostituendo la (30) nella (24) e semplificando per l'invertibilità di M si ottiene una dinamica dell'errore **lineare e disaccoppiata**:

$$\ddot{e} + K_d \dot{e} + K_p e + K_i \int e = 0 \quad (31)$$

Senza azione integrale ($K_i = 0$, impostazione di default), scegliendo $K_p = \omega_n^2 I$ e $K_d = 2\zeta\omega_n I$, ogni giunto si comporta come un sistema del secondo ordine con polinomio caratteristico $s^2 + 2\zeta\omega_n s + \omega_n^2$. Per ogni $K_p, K_d > 0$ l'errore converge a zero in modo esponenziale. Con i valori di default $\omega_n = 10$ rad/s e $\zeta = 1$ (smorzamento critico, senza sovraelongazione) il tempo di assestamento al 2% è circa $5.8/\omega_n \approx 0.6$ s.

Con l'azione integrale la (31) è del terzo ordine ed è stabile se $K_p, K_d, K_i > 0$ e $K_d K_p > K_i$. Una scelta tipica è porre tre poli coincidenti in $-\omega_n$: $K_p = 3\omega_n^2$, $K_d = 3\omega_n$, $K_i = \omega_n^3$. L'integrale serve a recuperare errori costanti dovuti a imprecisioni del modello.

9.2 Alternativa: PD con compensazione di gravità

$$\tau = K_p e + K_d \dot{e} + K_i \int e + \hat{G}(q) \quad (32)$$

Non richiede M e C ed è più robusto rispetto agli errori sull'inerzia. Per la **regolazione** a un punto fisso, con $K_i = 0$, la stabilità asintotica si dimostra con la funzione di Lyapunov $V = \frac{1}{2} \dot{q}^T M \dot{q} + \frac{1}{2} e^T K_p e$ e la proprietà di antisimmetria di $\dot{M} - 2C$. Nell'**inseguimento** lascia un errore residuo che diminuisce all'aumentare dei guadagni.

Nel PD i guadagni agiscono direttamente sulle coppie [N·m/rad] e vanno tarati giunto per giunto in base all'inerzia vista da ciascun giunto (per questo sono separati in `p.ctr1.pd`). Guadagni uniformi elevati su giunti distali con inerzia molto piccola rendono il sistema numericamente rigido (stiff) e rallentano molto ode45.

9.3 Robustezza e saturazione

- `p.ctr1.modelScale` moltiplica masse e inerzie del modello usato dal controllore, lasciando invariato il robot "reale" simulato. Permette di valutare l'effetto di un modello imperfetto.
- `p.ctr1.tauMax` satura le coppie. I limiti devono superare la coppia necessaria a sostenere il robot contro la gravità (per il giunto 1 circa 11 N·m con i parametri di default), altrimenti il braccio cade e l'errore diverge.

10. Simulazione con ode45

Lo stato del sistema in anello chiuso è $x = [q; \dot{q}; z] \in \mathbb{R}^{12}$, dove $z = \int e \, dt$ è lo stato dell'azione integrale. Le equazioni di stato, implementate in `robot_ode_4R`, sono:

$$\dot{q} = \dot{q}, \quad \ddot{q} = M(q)^{-1} (\tau - C(q, \dot{q}) \dot{q} - G(q)), \quad \dot{z} = e \quad (33)$$

M , C e G sono quelli del robot "reale" (`p.robot`), mentre τ è calcolata da `controller_4R` con il modello stimato. `simulate_4R` procede in quattro passi:

1. pre-calcola e valida la traiettoria (`build_reference_4R`);
2. imposta la condizione iniziale $q(0) = q_d(0) + q_{offset}$, $\dot{q}(0) = \dot{q}_0$, $z(0) = 0$;
3. integra con `ode45` sull'intervallo $[0, T + t_{Hold}]$ con le tolleranze `RelTol` e `AbsTol`, campionando l'uscita ogni `dtOut`;
4. ricalcola su ogni campione q_d , e , τ , la posa cartesiana reale (tramite FK) e l'errore cartesiano, e restituisce tutto nella struttura `res`.

11. Grafici, visualizzazione e animazione

11.1 Grafici richiesti: $q(t)$, $e(t)$, $\tau(t)$

`plot_results_4R(res, p)` produce quattro figure: posizioni di giunto reali sovrapposte ai riferimenti, errore di giunto ed errore cartesiano (in scala logaritmica), coppie ai giunti e percorso nel piano XY. Le figure seguenti si riferiscono alla configurazione di default: cerchio, coppia calcolata, errore iniziale di $\pm 5^\circ$ su ogni giunto.

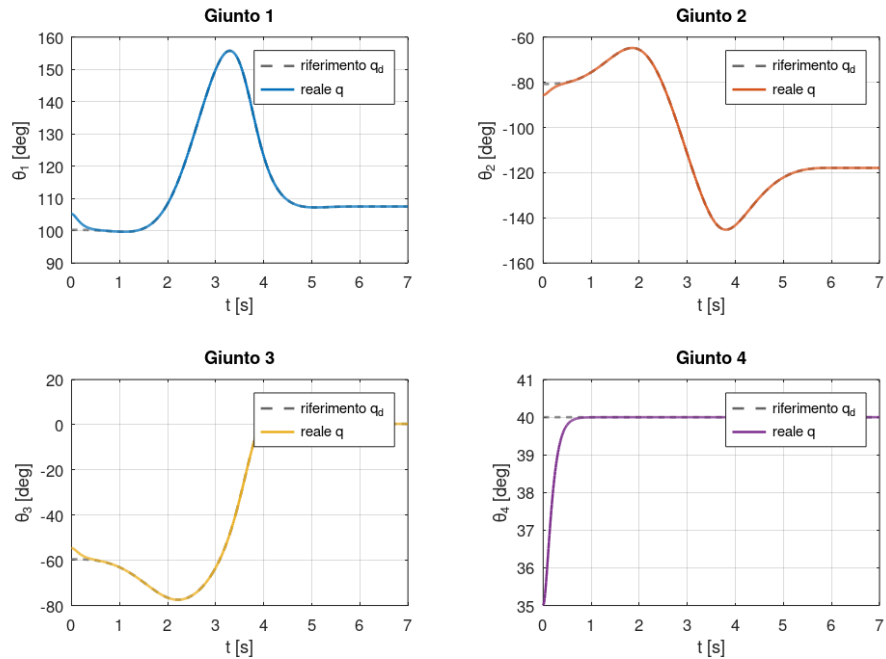


Figura 4 – Andamento di $q(t)$: riferimento tratteggiato e posizione reale.

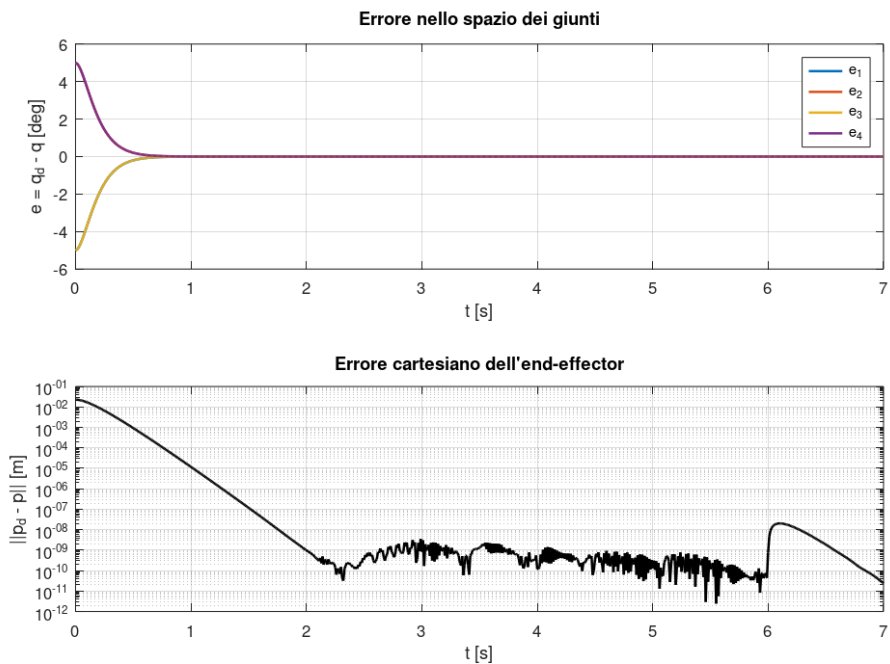


Figura 5 – Errore $e(t)$: l'errore iniziale di 5° si annulla in circa 0.6 s; l'errore cartesiano scende sotto 10^{-7} m.

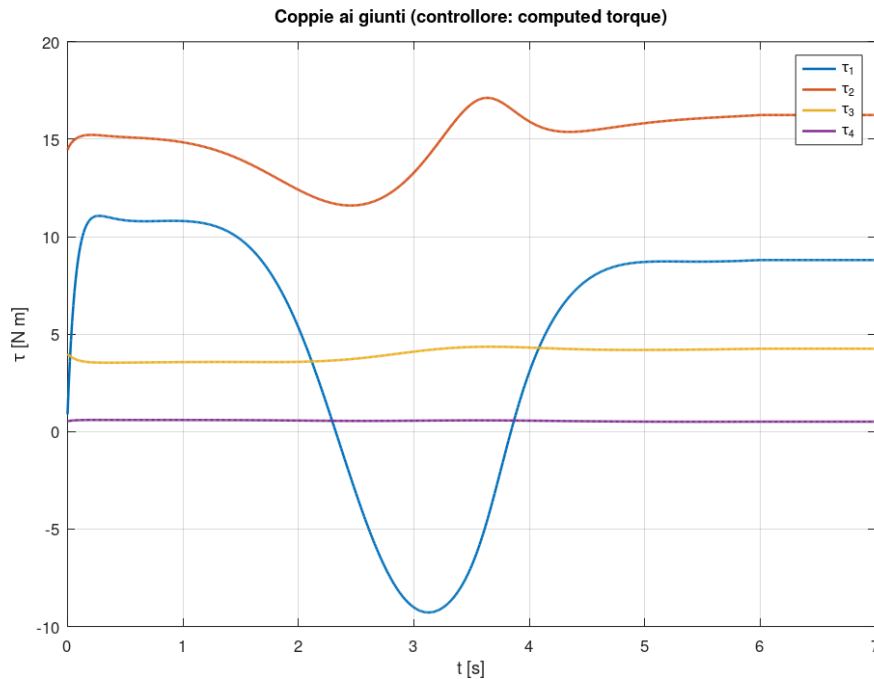


Figura 6 – Coppie $\tau(t)$: a robot fermo le coppie bilanciano la gravità.

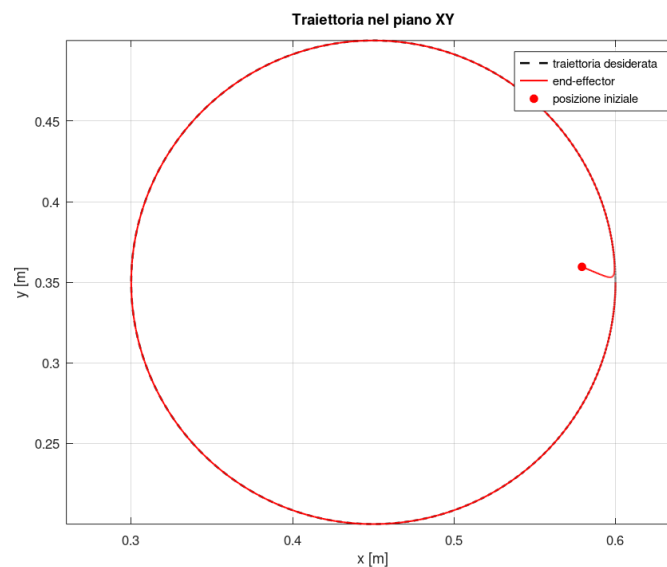


Figura 7 – Percorso nel piano XY: desiderato (tratteggiato) e reale (rosso).

11.2 Animazione

`animate_4R(res, p)` ridisegna il robot in ogni istante (un campione ogni `animStep`) aggiornando le proprietà degli oggetti grafici esistenti, che è molto più veloce che ricreare la figura, e chiama `drawnow`. In ogni fotogramma sono visibili:

- i link (blu), i giunti (cerchi bianchi), la base (triangolo) e l'end-effector (quadrato arancione);
- le **terne DH** 0...4 di ogni giunto, con asse x in rosso e asse y in verde (z è uscente dal piano), aggiornate a ogni istante;
- il **target**: il percorso desiderato completo (nero tratteggiato) e il punto desiderato all'istante corrente (cerchio verde);

- la scia percorsa dall'end-effector (rosso) e, nel titolo, il tempo e l'errore cartesiano in millimetri.

Con `p.viz.saveVideo = true` l'animazione viene salvata in MP4 tramite `VideoWriter` (solo MATLAB). L'animazione si può interrompere chiudendo la finestra.

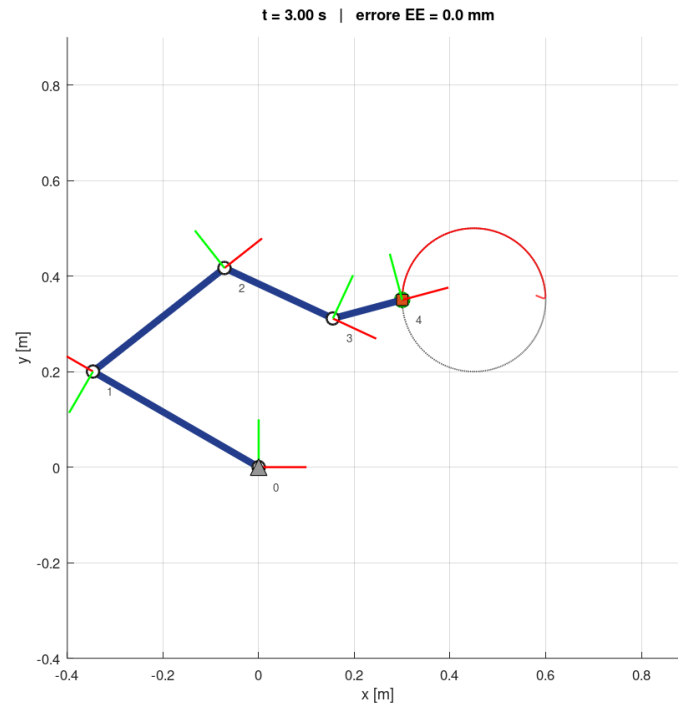


Figura 8 – Fotogramma dell'animazione a $t = 3$ s, con terne DH, target e scia dell'end-effector.

12. Test e verifica dei risultati

`ok = test_4R(p, runSymbolic)` esegue la seguente batteria di test, con generatore casuale inizializzato (`rng(1)`) per la riproducibilità. Stampa una tabella con errore misurato, soglia ed esito e restituisce `true` se tutti i test sono superati. Il secondo argomento facoltativo aggiunge il confronto con la derivazione simbolica.

#	Test	Errore misurato	Soglia	Esito
1	FK: x, y, φ coincidono con le formule (3)–(5); $\det R = 1$	$1.8e-15$	$1e-12$	PASS
2	IK: $FK(IK(p)) = p$, gomito alto e basso, 200 pose casuali	$1.8e-15$	$1e-9$	PASS
3	IK: la q originale è una delle due soluzioni	$5.6e-14$	$1e-7$	PASS
4	IK: $FK(IK(p)) = p$ con ψ automatico	$1.8e-15$	$1e-9$	PASS
5	IK: punto fuori dallo spazio di lavoro riconosciuto	0	0.5	PASS
6	Jacobiano e J rispetto alle differenze finite	$3.6e-10$	$1e-6$	PASS
7	$M(q)$ simmetrica	0	$1e-12$	PASS
8	$M(q)$ definita positiva (min autovalore $1.9e-3 > 0$)	$-1.9e-3$	0	PASS
9	$\dot{M} - 2C$ antisimmetrica	$1.2e-9$	$1e-6$	PASS
10	$G(q) = \partial V / \partial q$ (differenze finite su V)	$4.8e-9$	$1e-6$	PASS
11	Conservazione dell'energia con $\tau = 0$ per 2 s (J)	$2.6e-8$	$1e-5$	PASS
12	Coppia calcolata con modello esatto: errore cartesiano max (m)	$5.6e-9$	$1e-4$	PASS

Il test 11 è particolarmente significativo. Se M , C e G non fossero mutuamente coerenti, cioè non derivassero dalla stessa lagrangiana, l'energia totale $T + V$ in evoluzione libera non resterebbe costante. Una deriva di 10^{-8} J in 2 s, compatibile con l'errore numerico di `ode45`, conferma la correttezza del modello.

12.1 Risultati della simulazione in diversi scenari

Valori ottenuti con `simulate_4R` modificando un solo parametro rispetto ai default (errore cartesiano "a regime" = massimo per $t > T/2$):

Scenario	Errore finale [m]	Errore max a regime [m]	$ T $ max [N·m]
Default: cerchio, coppia calcolata	$2.4e-11$	$2.0e-8$	11.1 / 17.1 / 4.4 / 0.6
Segmento (<code>traj.type = 'line'</code>)	$4.6e-12$	$3.0e-9$	16.0 / 16.2 / 4.3 / 0.6
Gomito basso (<code>ik.elbow = 'down'</code>)	$2.4e-11$	$2.6e-9$	23.4 / 8.2 / 4.4 / 0.6
Modello errato +20% (<code>modelScale = 1.2</code>)	$1.8e-2$	$2.0e-2$	11.0 / 17.4 / 4.8 / 0.6
Modello errato +20% e $K_i = 200 \cdot I$	$3.5e-5$	$1.0e-2$	11.6 / 17.4 / 4.8 / 0.6

Scenario	Errore finale [m]	Errore max a regime [m]	$ \tau $ max [N·m]
PD + gravità (ctrl.type = 'pd_gravity')	6.0e-8	4.0e-3	17.5 / 32.6 / 5.7 / 1.5

Il confronto conferma la teoria. Con modello esatto la coppia calcolata insegue con errore praticamente nullo. Un errore di modello del 20% produce un errore di inseguimento di circa 2 cm, dovuto soprattutto alla compensazione imperfetta della gravità, che l'azione integrale riporta a zero una volta fermo il riferimento. Il PD + gravità, che non usa M e C, insegue con errore di qualche millimetro e richiede coppie maggiori durante i transitori.

13. Guida passo-passo per lanciare e testare il progetto

13.1 Prima esecuzione

1. **Preparazione:** estrarre **Robot4R.zip**, aprire MATLAB e posizionarsi nella cartella con **cd**, oppure con il pannello Current Folder. Tutti i file devono trovarsi nella stessa cartella.
2. **Verifica dell'installazione:** eseguire **test_4R**. In circa 5 secondi deve comparire la tabella dei test terminata da "TUTTI I TEST SUPERATI".
3. **Esecuzione completa:** eseguire **main_4R**. Nella Command Window compaiono tabella DH, matrici A_i e T , posa (x, y, ϕ) , le due soluzioni IK con il relativo errore, i valori di M , C e G , l'avanzamento della simulazione e gli errori finali. Si aprono le figure di FK, IK, $q(t)$, $e(t)$, $\tau(t)$, del percorso XY e l'animazione.
4. **Derivazione simbolica** (facoltativa): eseguire **S = derive_dynamics_4R_sym(params_4R(), true);**. Vengono stampati il residuo delle equazioni di Lagrange e lo scarto rispetto a **dynamics_4R**, entrambi dell'ordine di 10^{-15} , e generati i file nella cartella **generated**. **S.M**, **S.C** e **S.G** contengono le espressioni simboliche, da visualizzare per esempio con **pretty(S.M(1,1))**.

13.2 Scegliere i valori di prova

Aprire **main_4R.m** e, nella sezione PERSONALIZZAZIONE, decommentare o aggiungere le righe desiderate. Alcuni esempi pronti:

```
p = params_4R();
% --- robot diverso
p.robot.L = [0.5 0.4 0.3 0.15];
p.robot.m = [4 3 2 1];
p.robot.lc = p.robot.L/2; p.robot.I = p.robot.m.*p.robot.L.^2/12; % ricalcolare se cambiano L o m
% --- traiettoria rettilinea con rotazione dell'end-effector
p.traj.type = 'line'; p.traj.pStart = [0.6 -0.1]; p.traj.pEnd = [0.25 0.55];
p.traj.phiStart = 0; p.traj.phiEnd = deg2rad(45); p.traj.T = 4;
% --- cerchio piu' grande percorso due volte
p.traj.type = 'circle'; p.traj.radius = 0.2; p.traj.nTurns = 2; p.traj.T = 10;
% --- controllore e robustezza
p.ctrl.type = 'pd_gravity';
p.ctrl.modelScale = 1.2; p.ctrl.Ki = 200*eye(4);
wn = 20; p.ctrl.Kp = wn^2*eye(4); p.ctrl.Kd = 2*wn*eye(4); % risposta piu' rapida
% --- condizioni iniziali e grafica
p.sim.q0Offset = deg2rad([10 -10 10 -10]);
p.ik.elbow = 'down'; p.viz.animStep = 1; p.viz.saveVideo = true;
% --- eseguire solo alcune parti
p.run.fkDemo = false; p.run.ikDemo = false; p.run.animate = false;
```

Se si cambiano **L** o **m** in **main_4R.m**, ricalcolare anche **lc** e **I** come nell'esempio: in **params_4R.m** sono definiti a partire da **L** e **m**, ma i valori derivati non si aggiornano da soli dopo la creazione della struttura.

13.3 Usare le funzioni singolarmente

```
p = params_4R(); L = p.robot.L;
q = deg2rad([30 45 -30 20]);
[T, x, y, phi] = fk_4R(q, L) % cinematica diretta
theta = ik_4R(x, y, phi, L) % IK con psi automatico
[theta, info] = ik_4R(x, y, phi, L, 'down', deg2rad(25)); % gomito basso, psi = 25 deg
info.solutions % entrambe le soluzioni (colonne)
[~, xc, yc, phic] = fk_4R(theta, L); norm([xc-x, yc-y]) % verifica FK(ik(p))
[M, C, G] = dynamics_4R(q, zeros(4,1), p) % modello dinamico
figure; h = plot_robot_4R(gca, q, L); axis equal; grid on
res = simulate_4R(p); plot_results_4R(res, p); animate_4R(res, p);
```

13.4 Risoluzione dei problemi

Messaggio o sintomo	Causa	Soluzione
"Punto non raggiungibile a t = ..."	Un punto della traiettoria non è raggiungibile con il ψ scelto	Avvicinare la traiettoria alla base, ridurre il raggio o cambiare <code>p.ik.psiOffset</code> (es. $-40^\circ, 0^\circ, +60^\circ$)
Warning "vicino a una singolarità"	Durante il moto $ \sin \theta_2 < 0.05$	Cambiare <code>psiOffset</code> , il gomito o la traiettoria; vicino alla singolarità le velocità di giunto crescono molto
Simulazione molto lenta	Guadagni elevati su giunti con inerzia piccola (sistema stiff), tipico del PD	Ridurre i guadagni di <code>p.ctrl.pd</code> , tararli giunto per giunto, o allentare <code>RelTol/AbsTol</code>
Il robot "cade", errore enorme	<code>tauMax</code> inferiore alla coppia di gravità	Aumentare <code>p.ctrl.tauMax</code> o impostarlo a <code>Inf</code>
Errore costante a regime	Modello del controllore imperfetto (<code>modelScale</code> $\neq 1$)	Aumentare <code>Kp</code> o attivare l'azione integrale <code>Ki</code>
"Undefined function ..."	La cartella corrente non è quella del progetto	Eseguire <code>cd</code> nella cartella o <code>addpath(cartella)</code>
<code>derive_dynamics_4R_sym</code> non parte o è molto lenta	Symbolic Math Toolbox assente, oppure esecuzione in Octave	Eseguita in MATLAB con Symbolic Math Toolbox; per la simulazione basta <code>dynamics_4R</code> , che non richiede toolbox
Il video non viene salvato	In Octave <code>VideoWriter</code> non è disponibile	Usare MATLAB, oppure registrare lo schermo

13.5 Esecuzione in GNU Octave

Il codice usa solo funzioni comuni a MATLAB e Octave. Differenze note: il posizionamento automatico della legenda (`'best'`) produce un avviso innocuo; `VideoWriter` non è disponibile; la derivazione simbolica richiede `pkg install -forge symbolic` e SymPy ed è molto più lenta e pesante in memoria che in MATLAB (vedi Sezione 7.6).

Appendice A – Codice sorgente completo

I file sono riportati nell'ordine in cui conviene leggerli: parametri, cinematica, dinamica, traiettoria, controllo e simulazione, grafica, test e script principale. Gli stessi file sono contenuti nell'archivio [Robot4R.zip](#).

A.1 params_4R.m

```
function p = params_4R()
% PARAMS_4R Parametri configurabili del progetto "Robot manipolatore planare 4R".
%
% p = params_4R() restituisce una struttura con TUTTI i valori numerici
% usati dal progetto. Per provare configurazioni diverse e' sufficiente
% modificare i valori qui sotto, oppure sovrascrivere i campi della
% struttura nello script principale (main_4R.m) prima di lanciarlo, es.:
%
%     p = params_4R();
%     p.robot.L = [0.5 0.4 0.3 0.15];
%     p.traj.type = 'line';
%
% Unità di misura: SI (m, kg, kg*m^2, s, rad, N*m).

%% ===== PARAMETRI FISICI DEL ROBOT =====
p.robot.L = [0.40 0.35 0.25 0.15]; % lunghezze dei link L1..L4 [m]
p.robot.m = [3.0 2.5 1.5 0.8]; % masse dei link m1..m4 [kg]
% distanza del baricentro dal giunto a monte (default: metà link) [m]
p.robot.lc = p.robot.L / 2;
% momenti di inerzia baricentrici (default: asta sottile m*L^2/12) [kg m^2]
p.robot.I = p.robot.m .* p.robot.L.^2 / 12;
p.robot.g = 9.81; % accelerazione di gravità (verso -y) [m/s^2]

%% ===== CINEMATICA INVERSA =====
p.ik.elbow = 'up'; % configurazione del gomito: 'up' (alto) | 'down' (basso)
% Il 4R planare e' ridondante rispetto al compito (x, y, phi): serve un
% parametro aggiuntivo. Si usa psi = theta1+theta2+theta3 (orientamento
% assoluto del link 3). Durante la traiettoria: psi(t) = phi(t) + psiOffset.
p.ik.psiOffset = deg2rad(-40); % [rad] (0 => theta4 = 0)

%% ===== TRAIETTORIA DI RIFERIMENTO =====
p.traj.type = 'circle'; % 'circle' | 'line'
p.traj.T = 6; % durata del moto [s]
p.traj.tHold = 1; % tempo di permanenza finale (fermo) [s]
% --- cerchio ---
p.traj.center = [0.45 0.35]; % centro [xc yc] [m]
p.traj.radius = 0.15; % raggio [m]
p.traj.theta0 = 0; % angolo polare iniziale sul cerchio [rad]
p.traj.nTurns = 1; % numero di giri
% --- retta ---
p.traj.pStart = [0.60 -0.10]; % punto iniziale [x y] [m]
p.traj.pEnd = [0.25 0.55]; % punto finale [x y] [m]
% --- orientamento dell'end-effector (interpolato con la stessa legge oraria)
p.traj.phiStart = deg2rad(0); % [rad]
p.traj.phiEnd = deg2rad(30); % [rad]

%% ===== CONTROLLORE =====
p.ctrl.type = 'computed_torque'; % 'computed_torque' | 'pd_gravity'
% --- guadagni della coppia calcolata (agiscono sulle ACCELERAZIONI, 1/s^2 e 1/s)
% dinamica dell'errore: dde + Kd de + Kp e (+ Ki int(e)) = 0
wn = 10; % pulsazione naturale desiderata [rad/s]
zeta = 1; % smorzamento (1 = critico)
p.ctrl.Kp = diag(wn^2 * ones(1,4)); % [1/s^2]
p.ctrl.Kd = diag(2*zeta*wn * ones(1,4)); % [1/s]
p.ctrl.Ki = zeros(4); % [1/s^3] azione integrale (0 = disattivata)
% es. poli tripli in -wn: Kp = 3wn^2, Kd = 3wn, Ki = wn^3
% --- guadagni del PD + compensazione di gravità (agiscono sulle COPPIE)
% tarati giunto per giunto in base all'inerzia vista da ciascun giunto
p.ctrl.pd.Kp = diag([300 200 60 10]); % [N m/rad]
p.ctrl.pd.Kd = diag([40 25 6 0.5]); % [N m s/rad]
p.ctrl.pd.Ki = zeros(4); % [N m/(rad s)] azione integrale
% Fattore di stima del modello usato dal controllore (1 = modello perfetto).
% Es. 1.2 => il controllore crede che masse/inerzie siano +20% (test robustezza).
p.ctrl.modelScale = 1.0;
p.ctrl.tauMax = Inf; % saturazione coppie (scalare o 1x4) [N m]
```

```

%% ===== SIMULAZIONE =====
% Errore iniziale aggiunto alla configurazione desiderata q_d(0) [rad]
p.sim.q0Offset = deg2rad([5 -5 5 -5]);
p.sim.dq0 = [0 0 0 0]; % velocita' iniziali [rad/s]
p.sim.dtOut = 0.01; % passo di campionamento dei risultati [s]
p.sim.RelTol = 1e-6; % tolleranze ode45
p.sim.AbsTol = 1e-8;

%% ===== VISUALIZZAZIONE =====
p.viz.frameScale = 0.10; % lunghezza assi delle terne DH [m]
p.viz.animStep = 3; % disegna 1 campione ogni animStep
p.viz.pause = 0; % pausa aggiuntiva tra i frame [s]
p.viz.saveVideo = false; % true => salva animazione (VideoWriter)
p.viz.videoFile = 'robot4R_animazione.mp4';
p.viz.qDemo = deg2rad([30 45 -30 20]); % configurazione per il grafico FK

%% ===== FLAG DI ESECUZIONE (main_4R) =====
p.run.fkDemo = true; % grafico del robot in una configurazione
p.run.ikDemo = true; % verifica FK(IK(p)) e soluzioni gomito alto/basso
p.run.simulate = true; % simulazione ode45 con controllore
p.run.plots = true; % grafici q(t), e(t), tau(t)
p.run.animate = true; % animazione con drawnow
end

```

A.2 main_4R.m

```

%% MAIN_4R Robot manipolatore planare 4R con gravita' - script principale
%
% Esegue in sequenza (attivabili con i flag p.run.*):
% 1. cinematica diretta: tabella DH, matrici A_i, T, grafico del robot
% 2. cinematica inversa: soluzioni gomito alto/basso e verifica FK(IK(p))
% 3. dinamica: M, C, G nella configurazione di prova
% 4. simulazione ode45 del robot in anello chiuso lungo la traiettoria
% 5. grafici q(t), e(t), tau(t)
% 6. animazione con drawnow (terne DH e target)
%
% Tutti i valori numerici sono in params_4R.m: modificarli li' oppure
% sovrascriverli nella sezione "PERSONALIZZAZIONE" qui sotto.

clear; close all; clc;
p = params_4R();

%% ===== PERSONALIZZAZIONE (opzionale) =====
% Esempi (decommentare per provare):
% p.robot.L = [0.5 0.4 0.3 0.15];
% p.traj.type = 'line';
% p.ctrl.type = 'pd_gravity'; % PD + compensazione di gravita' (guadagni in p.ctrl.pd)
% p.ctrl.modelScale = 1.2; % modello del controllore errato del +20%
% p.ctrl.Ki = 200*eye(4); % azione integrale (recupera l'errore di modello)
% p.ik.elbow = 'down';
% p.run.animate = false;

L = p.robot.L;

%% 1. CINEMATICA DIRETTA
if p.run.fkDemo
    q = p.viz.qDemo(:);
    fprintf('=== CINEMATICA DIRETTA ===\n');
    DH = dh_table_4R(q, L);
    fprintf('Tabella DH [a_i alpha_i d_i theta_i(deg)]:\n');
    disp([DH(:,1:3) rad2deg(DH(:,4))]);
    for i = 1:4
        fprintf('A_%d =\n', i); disp(dh_matrix(DH(i,1), DH(i,2), DH(i,3), DH(i,4)));
    end
    [T, x, y, phi] = fk_4R(q, L);
    fprintf('T = A1*A2*A3*A4 =\n'); disp(T);
    fprintf('x = %.4f m, y = %.4f m, phi = %.2f deg\n\n', x, y, rad2deg(phi));

    figure('Name', 'Cinematica diretta', 'Color', 'w'); ax = gca;
    plot_robot_4R(ax, q, L, [], p.viz.frameScale);
    R = sum(L)*1.05; axis(ax, 'equal'); axis(ax, [-R R -R R]); grid(ax, 'on');
    xlabel('x [m]'); ylabel('y [m]');
    title(sprintf('q = [%s] deg -> (x, y, \phi) = (%.3f, %.3f, %.1f°)', ...
        num2str(rad2deg(q)), '%.0f ', x, y, rad2deg(phi)));
end

```

```

%% 2. CINEMATICA INVERSA
if p.run.ikDemo
    fprintf('=== CINEMATICA INVERSA ===\n');
    [~, x, y, phi] = fk_4R(p.viz.qDemo, L); % punto di prova raggiungibile
    psi = phi + p.ik.psiOffset;
    [~, info] = ik_4R(x, y, phi, L, 'up', psi);
    fprintf('Target (x, y, phi) = (%.4f, %.4f, %.2f deg), psi = %.2f deg\n', ...
        x, y, rad2deg(phi), rad2deg(psi));
    fprintf('Polso (xw, yw) = (%.4f, %.4f)\n', info.wrist);
    names = {'gomito alto ', 'gomito basso'};
    figure('Name', 'Cinematica inversa', 'Color', 'w');
    for k = 1:2
        qk = info.solutions(:,k);
        [~, xk, yk, phik] = fk_4R(qk, L);
        fprintf('%s: theta = [%s] deg | errore FK(IK) = %.2e\n', names{k}, ...
            num2str(rad2deg(qk), '%8.2f'), norm([xk-x, yk-y, mod(phik-phi+pi,2*pi)-pi]));
        ax = subplot(1,2,k);
        plot_robot_4R(ax, qk, L, [], p.viz.frameScale);
        plot(ax, x, y, 'go', 'MarkerSize', 14, 'LineWidth', 2);
        R = sum(L)*1.05; axis(ax, 'equal'); axis(ax, [-R R -R R]); grid(ax, 'on');
        title(ax, strtrim(names{k}));
    end
    fprintf('\n');
end

%% 3. DINAMICA (valori nella configurazione di prova)
fprintf('=== DINAMICA (q = qDemo, dq = 0.5 rad/s) ===\n');
[M, C, G] = dynamics_4R(p.viz.qDemo, 0.5*ones(4,1), p.robot);
fprintf('M(q) =\n'); disp(M);
fprintf('C(q,dq) =\n'); disp(C);
fprintf('G(q) =\n'); disp(G);

%% 4-6. SIMULAZIONE, GRAFICI E ANIMAZIONE
if p.run.simulate
    fprintf('=== SIMULAZIONE (%s, traiettoria %s) ===\n', p.ctrl.type, p.traj.type);
    res = simulate_4R(p);
    if p.run.plots, plot_results_4R(res, p); end
    if p.run.animate, animate_4R(res, p); end
end

```

A.3 dh_table_4R.m

```

function DH = dh_table_4R(theta, L)
% DH_TABLE_4R Tabella di Denavit-Hartenberg (convenzione standard) del 4R planare.
%
% DH = dh_table_4R(theta, L) restituisce una matrice 4x4 le cui righe
% sono i parametri [a_i alpha_i d_i theta_i] del giunto i:
%
%      i |  a_i  | alpha_i | d_i | theta_i
%      ---+-----+-----+-----+-----
%      1 |  L1  |      0   |  0  | theta1 (variabile)
%      2 |  L2  |      0   |  0  | theta2 (variabile)
%      3 |  L3  |      0   |  0  | theta3 (variabile)
%      4 |  L4  |      0   |  0  | theta4 (variabile)
%
% Tutti gli assi z_i sono paralleli (alpha_i = 0) e giacciono alla stessa
% quota (d_i = 0): il moto avviene interamente nel piano XY.
theta = theta(:); L = L(:);
DH = [L, zeros(4,1), zeros(4,1), theta];
end

```

A.4 dh_matrix.m

```

function A = dh_matrix(a, alpha, d, theta)
% DH_MATRIX Matrice di trasformazione omogenea A_i (convenzione DH standard).
%
% A = Rot_z(theta) * Trans_z(d) * Trans_x(a) * Rot_x(alpha)
%
%      [ cos(th)  -sin(th)*cos(al)  sin(th)*sin(al)  a*cos(th) ]
% A = [ sin(th)   cos(th)*cos(al)  -cos(th)*sin(al)  a*sin(th) ]
%      [  0         sin(al)         cos(al)         d         ]

```

```
%      [ 0      0      0      1 ]
ct = cos(theta); st = sin(theta);
ca = cos(alpha); sa = sin(alpha);
% (righe concatenate con vertcat: compatibile anche con variabili simboliche
% in GNU Octave)
A = vertcat([ ct, -st*ca, st*sa, a*ct ], ...
            [ st, ct*ca, -ct*sa, a*st ], ...
            [ 0, sa, ca, d ], ...
            [ 0, 0, 0, 1 ]);
end
```

A.5 fk_4R.m

```
function [T, x, y, phi, Tall] = fk_4R(theta, L)
% FK_4R Cinematica diretta del manipolatore planare 4R.
%
% [T, x, y, phi] = fk_4R(theta, L)
% [T, x, y, phi, Tall] = fk_4R(theta, L)
%
% Ingressi:
% theta : vettore 4x1 (o 1x4) delle variabili di giunto [rad]
% L      : vettore 4x1 (o 1x4) delle lunghezze dei link [m]
% Uscite:
% T       : matrice omogenea 4x4 T = A1*A2*A3*A4 (terna 4 rispetto alla base)
% x, y    : posizione dell'end-effector [m]
% phi     : orientamento dell'end-effector = theta1+theta2+theta3+theta4 [rad]
% Tall    : array 4x4x5 con le terne T0^0 (identita'), T0^1, ..., T0^4
%          (utile per disegnare il robot e le terne DH)
DH = dh_table_4R(theta, L);
Tall = zeros(4,4,5);
Tall(:, :, 1) = eye(4);
T = eye(4);
for i = 1:4
    A = dh_matrix(DH(i,1), DH(i,2), DH(i,3), DH(i,4));
    T = T * A;
    Tall(:, :, i+1) = T;
end
x = T(1,4);
y = T(2,4);
phi = atan2(T(2,1), T(1,1)); % = theta1+theta2+theta3+theta4 (ricondotto in (-pi, pi))
end
```

A.6 ik_4R.m

```
function [theta, info] = ik_4R(x, y, phi, L, elbow, psi)
% IK_4R Cinematica inversa analitica del manipolatore planare 4R.
%
% theta      = ik_4R(x, y, phi, L)
% theta      = ik_4R(x, y, phi, L, elbow)
% theta      = ik_4R(x, y, phi, L, elbow, psi)
% [theta,info] = ik_4R(...)
%
% Ingressi:
% x, y      : posizione desiderata dell'end-effector [m]
% phi       : orientamento desiderato phi = th1+th2+th3+th4 [rad]
% L         : lunghezze dei link [L1 L2 L3 L4] [m]
% elbow     : 'up' (gomito alto, default) | 'down' (gomito basso)
%            (accetta anche 'alto' / 'basso')
% psi       : parametro di ridondanza psi = th1+th2+th3, cioe' l'orientamento
%            assoluto del link 3 [rad]. Se omissso o vuoto si usa psi = phi
%            (theta4 = 0); se tale scelta non e' raggiungibile si cerca il
%            valore di psi raggiungibile piu' vicino a phi.
% Uscite:
% theta     : vettore 4x1 [th1; th2; th3; th4] (NaN se il punto non e'
%            raggiungibile)
% info      : struttura con
%            .reachable true/false
%            .psi       valore di psi effettivamente usato
%            .wrist     [xw yw] punto del "polso" (a monte del link 4)
%            .p2        [x2 y2] punto a valle del link 2
%            .solutions 4x2, colonne = soluzione gomito alto / gomito basso
%
```

```

% Procedura:
% 1) polso: xw = x - L4 cos(phi), yw = y - L4 sin(phi)
% 2) gruppo 3R (th1,th2,th3) con orientamento psi del link 3:
%      x2 = xw - L3 cos(psi), y2 = yw - L3 sin(psi)
%      problema 2R: c2 = (x2^2+y2^2-L1^2-L2^2)/(2 L1 L2)
%                  s2 = -/+ sqrt(1-c2^2) (gomito alto / basso)
%                  th2 = atan2(s2, c2)
%                  th1 = atan2(y2,x2) - atan2(L2 s2, L1 + L2 c2)
%                  th3 = psi - th1 - th2
% 3) quarto giunto: th4 = phi - (th1 + th2 + th3) = phi - psi
if nargin < 5 || isempty(elbow), elbow = 'up'; end
L = L(:);
switch lower(elbow)
    case {'up', 'alto'}, sel = 1;
    case {'down', 'basso'}, sel = 2;
    otherwise, error('ik_4R:elbow', 'elbow deve essere ''up'' o ''down''.');
end

% 1) punto del polso
xw = x - L(4)*cos(phi);
yw = y - L(4)*sin(phi);

% 2) scelta del parametro di ridondanza psi
if nargin < 6 || isempty(psi)
    psi = phi;
    if ~reach2R(xw - L(3)*cos(psi), yw - L(3)*sin(psi), L)
        cand = phi + linspace(-pi, pi, 721);
        ok = false(size(cand));
        for k = 1:numel(cand)
            ok(k) = reach2R(xw - L(3)*cos(cand(k)), yw - L(3)*sin(cand(k)), L);
        end
        if any(ok)
            cand = cand(ok);
            [~, idx] = min(abs(cand - phi));
            psi = cand(idx);
        end
    end
end

% 3) problema 2R per theta1, theta2
x2 = xw - L(3)*cos(psi);
y2 = yw - L(3)*sin(psi);
c2 = (x2^2 + y2^2 - L(1)^2 - L(2)^2) / (2*L(1)*L(2));

info.psi = psi;
info.wrist = [xw yw];
info.p2 = [x2 y2];

if abs(c2) > 1 + 1e-12
    info.reachable = false;
    info.solutions = nan(4,2);
    theta = nan(4,1);
    return
end
c2 = max(min(c2, 1), -1);
s2v = [-sqrt(1 - c2^2), +sqrt(1 - c2^2)]; % [gomito alto, gomito basso]
sols = zeros(4,2);
for k = 1:2
    s2 = s2v(k);
    th2 = atan2(s2, c2);
    th1 = atan2(y2, x2) - atan2(L(2)*s2, L(1) + L(2)*c2);
    th3 = psi - th1 - th2;
    th4 = phi - psi;
    sols(:,k) = wrapPi([th1; th2; th3; th4]);
end
info.reachable = true;
info.solutions = sols;
theta = sols(:, sel);
end

function ok = reach2R(x2, y2, L)
r = hypot(x2, y2);
ok = (r <= L(1) + L(2)) && (r >= abs(L(1) - L(2)));
end

function a = wrapPi(a)
a = mod(a + pi, 2*pi) - pi;
end

```

A.7 jacobian_4R.m

```
function [J, Jdot] = jacobian_4R(theta, L, dtheta)
% JACOBIAN_4R Jacobiano analitico (planare) del 4R e sua derivata temporale.
%
% [J, Jdot] = jacobian_4R(theta, L, dtheta)
%
% J (3x4) lega le velocita' di giunto alle velocita' del compito:
% [dx; dy; dphi] = J(theta) * dtheta
% con, detti phi_k = theta1+...+thetak gli angoli assoluti dei link,
% J(1,j) = -sum_{k>=j} L_k sin(phi_k)
% J(2,j) = sum_{k>=j} L_k cos(phi_k)
% J(3,j) = 1
% Jdot (3x4) e' la derivata temporale di J (richiede dtheta).
theta = theta(:); L = L(:);
if nargin < 3, dtheta = zeros(4,1); end
dtheta = dtheta(:);
phiAbs = cumsum(theta); % angoli assoluti dei link
dphiAbs = cumsum(dtheta);
J = zeros(3,4); Jdot = zeros(3,4);
for j = 1:4
    k = j:4;
    J(1,j) = -sum(L(k) .* sin(phiAbs(k)));
    J(2,j) = sum(L(k) .* cos(phiAbs(k)));
    J(3,j) = 1;
    Jdot(1,j) = -sum(L(k) .* cos(phiAbs(k)) .* dphiAbs(k));
    Jdot(2,j) = -sum(L(k) .* sin(phiAbs(k)) .* dphiAbs(k));
end
end
```

A.8 dynamics_4R.m

```
function [M, C, G] = dynamics_4R(theta, dtheta, params)
% DYNAMICS_4R Modello dinamico (Eulero-Lagrange) del manipolatore planare 4R.
%
% [M, C, G] = dynamics_4R(theta, dtheta, params)
%
% M(q) ddq + C(q,dq) dq + G(q) = tau
%
% Ingressi:
% theta, dtheta : posizioni [rad] e velocita' [rad/s] di giunto (4x1)
% params        : struttura con i campi L, lc, m, I, g (vedi params_4R:
%                  si puo' passare p.robot). Se params e' la struttura
%                  completa p, viene usato automaticamente p.robot.
%
% Uscite:
% M : matrice di inerzia 4x4 (simmetrica, definita positiva)
% C : matrice di Coriolis/centrifughi 4x4 (simboli di Christoffel,
%      quindi dM/dt - 2C e' antisimmetrica)
% G : vettore gravitazionale 4x1
%
% Formulazione chiusa (verificata contro la derivazione simbolica in
% derive_dynamics_4R_sym.m). Con phi_k = th1+...+thk (angoli assoluti),
% a_ik = L_k se k < i, a_ik = lc_i se k = i, e S matrice triangolare
% inferiore di 1 (dphi = S*dq):
% B_kl = sum_{i>=max(k,l)} m_i a_ik a_il
% Mb_kl = B_kl cos(phi_k - phi_l) + delta_kl I_k      -> M = S' Mb S
% Cb_kl = B_kl sin(phi_k - phi_l) dphi_l              -> C = S' Cb S
% Gb_k = g cos(phi_k) sum_{i>=k} m_i a_ik              -> G = S' Gb
if isfield(params, 'robot'), params = params.robot; end
L = params.L(:); lc = params.lc(:); m = params.m(:); I = params.I(:); g = params.g;
theta = theta(:); dtheta = dtheta(:);
n = 4;
S = tril(ones(n));
ph = S*theta; % angoli assoluti
dph = S*dtheta;

% matrice dei "bracci" a(i,k)
a = zeros(n);
for i = 1:n
    a(i,1:i-1) = L(1:i-1).';
    a(i,i) = lc(i);
end
```

```
% B_kl = sum_i m_i a_ik a_il   (a_ik = 0 per k > i)
B = a.' * diag(m) * a;

D = ph - ph.';                % D(k,l) = phi_k - phi_l
Mb = B .* cos(D) + diag(I);
Cb = (B .* sin(D)) .* (ones(n,1) * dph.'); % Cb(k,l) = B_kl sin(.) dphi_l
Gb = g * cos(ph) .* (a.' * m);

M = S.' * Mb * S;
C = S.' * Cb * S;
G = S.' * Gb;
M = (M + M.')/2; % simmetria numerica
end
```

A.9 energy_4R.m

```
function [Ekin, Epot] = energy_4R(theta, dtheta, params)
% ENERGY_4R Energia cinetica e potenziale del 4R (per verifiche numeriche).
%
%   Ekin = 1/2 dq' M(q) dq
%   Epot = sum_i m_i g y_ci(q)   (y_ci quota del baricentro del link i)
if isfield(params, 'robot'), params = params.robot; end
theta = theta(:); dtheta = dtheta(:);
M = dynamics_4R(theta, zeros(4,1), params);
Ekin = 0.5 * dtheta.' * M * dtheta;
ph = cumsum(theta);
L = params.L(:); lc = params.lc(:); m = params.m(:);
Epot = 0;
for i = 1:4
    yci = sum(L(1:i-1) .* sin(ph(1:i-1))) + lc(i)*sin(ph(i));
    Epot = Epot + m(i) * params.g * yci;
end
end
```

A.10 derive_dynamics_4R_sym.m

```
function S = derive_dynamics_4R_sym(p, doGenerate)
% DERIVE_DYNAMICS_4R_SYM Derivazione simbolica (Eulero-Lagrange) di M, C, G.
%
%   S = derive_dynamics_4R_sym(p)
%   S = derive_dynamics_4R_sym(p, doGenerate)
%
%   Richiede Symbolic Math Toolbox (MATLAB) oppure il pacchetto "symbolic"
%   (GNU Octave). Passi eseguiti:
%   1) posizioni dei baricentri dalle matrici DH simboliche
%   2) energia cinetica  $T = \sum_i 1/2 m_i |v_{ci}|^2 + 1/2 I_i \omega_i^2$ 
%   3) energia potenziale  $V = \sum_i m_i g y_{ci}$ 
%   4)  $M(q) = d^2 T / d dq^2$ ,  $G(q) = dV/dq$ 
%   5)  $C(q, dq)$  tramite simboli di Christoffel
%   6) verifica: le equazioni di Lagrange  $d/dt(dL/d dq) - dL/dq$  calcolate
%   direttamente coincidono con  $M ddq + C dq + G$ 
%   7) confronto numerico con dynamics_4R (formulazione chiusa) usando i
%   valori in p (default: params_4R)
%   8) (solo MATLAB, se doGenerate = true) genera con matlabFunction i file
%   M_4R_sym.m, C_4R_sym.m, G_4R_sym.m nella cartella "generated"
%
%   S contiene le espressioni simboliche (S.M, S.C, S.G, S.T, S.V) e
%   l'errore massimo del confronto numerico (S.maxErr).
if nargin < 1 || isempty(p), p = params_4R(); end
if nargin < 2, doGenerate = false; end
isOctave = exist('OCTAVE_VERSION', 'builtin') ~= 0;
if isOctave, pkg load symbolic; end
% semplificazione delle espressioni: rapida in MATLAB, molto lenta in Octave
if isOctave, simp = @(e) e; else, simp = @(e) simplify(e); end

syms q1 q2 q3 q4 dq1 dq2 dq3 dq4 ddq1 ddq2 ddq3 ddq4 real
syms L1 L2 L3 L4 lc1 lc2 lc3 lc4 m1 m2 m3 m4 I1 I2 I3 I4 g real
q = [q1; q2; q3; q4]; dq = [dq1; dq2; dq3; dq4]; ddq = [ddq1; ddq2; ddq3; ddq4];
L = [L1; L2; L3; L4]; lc = [lc1; lc2; lc3; lc4];
m = [m1; m2; m3; m4]; I = [I1; I2; I3; I4];

% 1) terne DH e baricentri
```

```

T = sym(eye(4));
o = sym(zeros(3,5));          % origini delle terne 0..4
for i = 1:4
    T = T * dh_matrix(L(i), 0, 0, q(i));
    o(:, i+1) = T(1:3, 4);
end
Tkin = sym(0); V = sym(0);
for i = 1:4
    pc = o(:,i) + lc(i)/L(i) * (o(:,i+1) - o(:,i)); % baricentro del link i
    Jv = jacobian(pc, q);
    vc = Jv * dq;
    w = sum(dq(1:i)); % velocita' angolare (asse z)
    Tkin = Tkin + m(i)*(vc.*vc)/2 + I(i)*w^2/2;
    V = V + m(i)*g*pc(2); % gravita' lungo -y
end

fprintf(' [1-3] energie T e V calcolate\n');
% 4) matrice di inerzia e vettore gravitazionale
M = sym(zeros(4));
for k = 1:4
    for j = 1:4
        M(k,j) = simp(diff(diff(Tkin, dq(k)), dq(j)));
    end
end
G = sym(zeros(4,1));
for k = 1:4, G(k) = simp(diff(V, q(k))); end

fprintf(' [4] M(q) e G(q) calcolati\n');
% 5) Coriolis / centrifughi con i simboli di Christoffel
% C_kj = sum_i 1/2 ( dM_kj/dq_i + dM_ki/dq_j - dM_ij/dq_k ) dq_i
% In forma matriciale, detta D_i = dM/dq_i e V = [D_1 dq, ..., D_4 dq]:
% C = 1/2 ( dM/dt + V - V' ), con dM/dt = sum_i D_i dq_i
% (equivalente al triplo ciclo ma con molte meno operazioni simboliche)
dMdq = jacobian(M(:), q); % 16x4: colonna i = vec(dM/dq_i)
Mdot = sym(zeros(4)); Vm = sym(zeros(4));
for i = 1:4
    Di = reshape(dMdq(:, i), 4, 4);
    Mdot = Mdot + Di*dq(i);
    Vm(:, i) = Di*dq;
end
C = (Mdot + Vm - Vm.)/2;
fprintf(' [5] C(q,dq) calcolata (Christoffel)\n');
% 6) verifica delle equazioni di Lagrange calcolate direttamente
Lag = Tkin - V;
tauLag = sym(zeros(4,1));
for k = 1:4
    dLdq_dot = diff(Lag, dq(k));
    ddt = jacobian(dLdq_dot, q)*dq + jacobian(dLdq_dot, dq)*ddq; % derivata totale
    tauLag(k) = ddt - diff(Lag, q(k));
end
resid = tauLag - (M*ddq + C*dq + G);

% 7) confronto numerico con dynamics_4R
r = p.robot;
symVars = [L1 L2 L3 L4 lc1 lc2 lc3 lc4 m1 m2 m3 m4 I1 I2 I3 I4 g];
numVals = [r.L(:).' r.lc(:).' r.m(:).' r.I(:).' r.g];
maxErr = 0; maxRes = 0;
for trial = 1:3
    qv = 2*pi*rand(4,1) - pi; dqv = 2*rand(4,1) - 1; ddqv = 2*rand(4,1) - 1;
    vars = [symVars, q.', dq.', ddq.'];
    vals = [numVals, qv.', dqv.', ddqv.'];
    Ms = double(subs(M, vars, vals));
    Cs = double(subs(C, vars, vals));
    Gs = double(subs(G, vars, vals));
    Rs = double(subs(resid, vars, vals));
    [Mn, Cn, Gn] = dynamics_4R(qv, dqv, r);
    maxErr = max([maxErr, max(abs(Ms(:))-Mn(:))], max(abs(Cs(:))-Cn(:)), max(abs(Gs-Gn)));
    maxRes = max(maxRes, max(abs(Rs)));
end
fprintf('Derivazione simbolica completata.\n');
fprintf(' residuo Lagrange diretto vs M ddot q + C dq + G : %.3e\n', maxRes);
fprintf(' max |simbolico - dynamics_4R| : %.3e\n', maxErr);

% 8) generazione automatica di funzioni (solo MATLAB)
if doGenerate
    if isOctave
        warning('Generazione file con matlabFunction disponibile solo in MATLAB.');
```

```

        if ~exist(outDir, 'dir'), mkdir(outDir); end
        matlabFunction(M, 'File', fullfile(outDir, 'M_4R_sym'), 'Vars', {q, L, lc, m, I});
        matlabFunction(C, 'File', fullfile(outDir, 'C_4R_sym'), 'Vars', {q, dq, L, lc, m, I});
        matlabFunction(G, 'File', fullfile(outDir, 'G_4R_sym'), 'Vars', {q, L, lc, m, g});
        fprintf(' file generati in %s\n', outDir);
    end
end
S = struct('M', M, 'C', C, 'G', G, 'T', Tkin, 'V', V, 'maxErr', maxErr, 'maxResLagrange', maxRes);
end

```

A.11 task_reference_4R.m

```

function [X, dX, ddX] = task_reference_4R(t, p)
% TASK_REFERENCE_4R Traiettorie desiderata nello spazio operativo (aumentato).
%
% [X, dX, ddX] = task_reference_4R(t, p)
%
% X = [x; y; phi; psi] con derivate prima e seconda (analitiche).
% - percorso geometrico: cerchio o segmento (p.traj.type)
% - legge oraria: polinomio quintico s(t) in [0,1] su [0, T], con
%   velocità e accelerazione nulle agli estremi; dopo T il punto resta fermo
% - orientamento: phi(s) = phiStart + (phiEnd - phiStart) s
% - ridondanza: psi = phi + p.ik.psiOffset (psi = th1+th2+th3)
[s, ds, dds] = quintic_s(t, p.traj.T);
switch lower(p.traj.type)
    case 'circle'
        c = p.traj.center(:); r = p.traj.radius;
        k = 2*pi*p.traj.nTurns;
        a = p.traj.theta0 + k*s; da = k*ds; dda = k*dds;
        pos = c + r*[cos(a); sin(a)];
        vel = r*[-sin(a); cos(a)]*da;
        acc = r*[-cos(a); -sin(a)]*da^2 + [-sin(a); cos(a)]*dda;
    case 'line'
        p0 = p.traj.pStart(:); p1 = p.traj.pEnd(:);
        pos = p0 + (p1 - p0)*s;
        vel = (p1 - p0)*ds;
        acc = (p1 - p0)*dds;
    otherwise
        error('task_reference_4R:type', 'p.traj.type deve essere ''circle'' o ''line''.');
end
dphi = p.traj.phiEnd - p.traj.phiStart;
phi = p.traj.phiStart + dphi*s;
X = [pos; phi; phi + p.ik.psiOffset];
dX = [vel; dphi*ds; dphi*ds];
ddX = [acc; dphi*dds; dphi*dds];
end

function [s, ds, dds] = quintic_s(t, T)
% legge oraria quintica: s = 10 tau^3 - 15 tau^4 + 6 tau^5, tau = t/T
if t <= 0
    s = 0; ds = 0; dds = 0; return
elseif t >= T
    s = 1; ds = 0; dds = 0; return
end
tau = t/T;
s = 10*tau^3 - 15*tau^4 + 6*tau^5;
ds = (30*tau^2 - 60*tau^3 + 30*tau^4) / T;
dds = (60*tau - 180*tau^2 + 120*tau^3) / T^2;
end

```

A.12 build_reference_4R.m

```

function ref = build_reference_4R(p)
% BUILD_REFERENCE_4R Pre-calcola e valida la traiettoria di riferimento.
%
% ref = build_reference_4R(p)
%
% Campiona la traiettoria cartesiana su una griglia fine, risolve la
% cinematica inversa in ogni punto (gomito p.ik.elbow, psi da p.traj),
% verifica che tutti i punti siano raggiungibili e lontani da singolarità
% e "srotola" (unwrap) gli angoli per avere q_d(t) continua.
% La struttura ref e' poi usata da reference_4R durante la simulazione.

```

```

L = p.robot.L;
ref.tEnd = p.traj.T + p.traj.tHold;
ref.t = linspace(0, ref.tEnd, max(2001, ceil(ref.tEnd/1e-3)+1));
N = numel(ref.t);
ref.q = zeros(4,N); ref.X = zeros(4,N); ref.sin2 = zeros(1,N);
for k = 1:N
    X = task_reference_4R(ref.t(k), p);
    [q, info] = ik_4R(X(1), X(2), X(3), L, p.ik.elbow, X(4));
    if ~info.reachable
        error('build_reference_4R:unreachable', ...
            ['Punto non raggiungibile a t = %.3f s: (x,y,phi) = (%.3f, %.3f, %.1f deg).\n' ...
            'Modificare la traiettoria (p.traj) o psiOffset (p.ik.psiOffset).'], ...
            ref.t(k), X(1), X(2), rad2deg(X(3)));
    end
    ref.q(:,k) = q; ref.X(:,k) = X; ref.sin2(k) = abs(sin(q(2)));
end
ref.q = unwrap(ref.q, [], 2);
if min(ref.sin2) < 0.05
    warning('build_reference_4R:singular', ...
        'La traiettoria passa vicino a una singolarita' (|sin th2| min = %.3f).', min(ref.sin2));
end
end

```

A.13 reference_4R.m

```

function [qd, dqd, ddqd, Xd] = reference_4R(t, ref, p)
% REFERENCE_4R Riferimento nello spazio dei giunti all'istante t.
%
% [qd, dqd, ddqd, Xd] = reference_4R(t, ref, p)
%
% qd : da cinematica inversa analitica (ik_4R), riportata sul ramo
%      continuo pre-calcolato in ref (multipli di 2*pi)
% dqd : Ja^-1 * dX (Jacobiano aumentato 4x4)
% ddqd : Ja^-1 * (ddX - dJa * dqd)
% dove Ja = [J(1:2,:); 1 1 1 1; 1 1 1 0] lega dq a [dx dy dphi dpsi].
L = p.robot.L;
t = min(max(t, 0), ref.tEnd);
[Xd, dX, ddX] = task_reference_4R(t, p);
qd = ik_4R(Xd(1), Xd(2), Xd(3), L, p.ik.elbow, Xd(4));
qGrid = interp1(ref.t, ref.q.', t).';
qd = qd + 2*pi*round((qGrid - qd)/(2*pi)); % stesso ramo continuo
[J, ~] = jacobian_4R(qd, L);
Ja = [J(1:2,:); 1 1 1 1; 1 1 1 0];
dqd = Ja \ dX;
[~, Jdot] = jacobian_4R(qd, L, dqd);
dJa = [Jdot(1:2,:); zeros(2,4)];
ddqd = Ja \ (ddX - dJa*dqd);
end

```

A.14 controller_4R.m

```

function [tau, qd, dqd, e, de] = controller_4R(t, q, dq, z, ref, p)
% CONTROLLER_4R Legge di controllo nello spazio dei giunti.
%
% [tau, qd, dqd, e, de] = controller_4R(t, q, dq, z, ref, p)
%
% e = qd - q, de = dqd - dq, z = integrale di e (stato del controllore)
%
% p.ctrl.type = 'computed_torque' (coppia calcolata / inverse dynamics):
%   tau = M^(q) (ddqd + Kd de + Kp e + Ki z) + C^(q,dq) dq + G^(q)
%   Con modello perfetto la dinamica dell'errore diventa lineare e
%   disaccoppiata: dde + Kd de + Kp e + Ki z = 0 (esponenzialmente
%   stabile se Kp, Kd > 0 e, con azione integrale, Kd Kp > Ki).
%   Guadagni: p.ctrl.Kp, p.ctrl.Kd, p.ctrl.Ki.
%
% p.ctrl.type = 'pd_gravity' (PD con compensazione di gravita'):
%   tau = Kp e + Kd de + Ki z + G^(q)
%   Stabile (Lyapunov) per la regolazione; nell'inseguimento lascia un
%   errore residuo che diminuisce all'aumentare dei guadagni.
%   Guadagni: p.ctrl.pd.Kp, p.ctrl.pd.Kd, p.ctrl.pd.Ki.
%
% M^, C^, G^ sono calcolati con masse e inerzie moltiplicate per

```

```
% p.ctrl.modelScale (1 = modello esatto) per testare la robustezza.
q = q(:); dq = dq(:); z = z(:);
[qd, ddq, dddq] = reference_4R(t, ref, p);
e = qd - q;
de = ddq - dq;

rHat = p.robot;
rHat.m = rHat.m * p.ctrl.modelScale;
rHat.I = rHat.I * p.ctrl.modelScale;

switch lower(p.ctrl.type)
    case 'computed_torque'
        [Mh, Ch, Gh] = dynamics_4R(q, dq, rHat);
        tau = Mh*(ddq + p.ctrl.Kd*de + p.ctrl.Kp*e + p.ctrl.Ki*z) + Ch*dq + Gh;
    case 'pd_gravity'
        [~, ~, Gh] = dynamics_4R(q, dq, rHat);
        tau = p.ctrl.pd.Kp*e + p.ctrl.pd.Kd*de + p.ctrl.pd.Ki*z + Gh;
    otherwise
        error('controller_4R:type', 'p.ctrl.type deve essere ''computed_torque'' o ''pd_gravity''.');
end
tauMax = p.ctrl.tauMax(:);
tau = max(min(tau, tauMax), -tauMax);
end
```

A.15 robot_ode_4R.m

```
function dx = robot_ode_4R(t, x, ref, p)
% ROBOT_ODE_4R Equazioni di stato del robot in anello chiuso (per ode45).
%
% Stato x = [q; dq; z] (12x1), con z = integrale dell'errore e = qd - q
% (stato dell'azione integrale del controllore).
% ddq = M(q) \ ( tau - C(q,dq) dq - G(q) ), dz = e
% con tau fornita da controller_4R e M, C, G del robot "reale" (p.robot).
q = x(1:4);
dq = x(5:8);
z = x(9:12);
[tau, ~, ~, e] = controller_4R(t, q, dq, z, ref, p);
[M, C, G] = dynamics_4R(q, dq, p.robot);
ddq = M \ (tau - C*dq - G);
dx = [dq; ddq; e];
end
```

A.16 simulate_4R.m

```
function res = simulate_4R(p)
% SIMULATE_4R Simula il robot 4R in anello chiuso con ode45.
%
% res = simulate_4R(p)
%
% Uscita res (campionata ogni p.sim.dtOut):
%   res.t           tempi (1xN)
%   res.q, res.dq    posizioni / velocita' di giunto (4xN)
%   res.qd, res.ddq  riferimenti di giunto (4xN)
%   res.e            errore di posizione qd - q (4xN)
%   res.tau          coppie ai giunti (4xN)
%   res.Xd           riferimento cartesiano [x;y;phi;psi] (4xN)
%   res.Xee          posa reale dell'end-effector [x;y;phi] (3xN)
%   res.ref          struttura della traiettoria pre-calcolata
ref = build_reference_4R(p);
q0 = ref.q(:,1) + p.sim.q0Offset(:);
x0 = [q0; p.sim.dq0(:); zeros(4,1)]; % [q; dq; integrale errore]
tspan = 0:p.sim.dtOut:ref.tEnd;
opts = odeset('RelTol', p.sim.RelTol, 'AbsTol', p.sim.AbsTol);
fprintf('Simulazione ode45 su [0, %.2f] s ... ', ref.tEnd);
tic;
[t, X] = ode45(@(t,x) robot_ode_4R(t, x, ref, p), tspan, x0, opts);
fprintf('completata in %.1f s.\n', toc);

N = numel(t);
res.t = t(:).'; res.q = X(:,1:4).'; res.dq = X(:,5:8).'; res.z = X(:,9:12).';
res.qd = zeros(4,N); res.ddq = zeros(4,N); res.e = zeros(4,N);
```

```

res.tau = zeros(4,N); res.Xd = zeros(4,N); res.Xee = zeros(3,N);
for k = 1:N
    [tau, qd, dqd, e] = controller_4R(t(k), res.q(:,k), res.dq(:,k), res.z(:,k), ref, p);
    res.tau(:,k) = tau; res.qd(:,k) = qd; res.dqd(:,k) = dqd; res.e(:,k) = e;
    res.Xd(:,k) = task_reference_4R(t(k), p);
    [~, x, y] = fk_4R(res.q(:,k), p.robot.L);
    res.Xee(:,k) = [x; y; sum(res.q(:,k))];
end
res.ref = ref;
ex = res.Xd(1:2,:) - res.Xee(1:2,:);
res.cartErr = sqrt(sum(ex.^2, 1));
fprintf(' errore cartesiano finale: %.2e m | max dopo il transitorio (t > T/2): %.2e m\n', ...
    res.cartErr(end), max(res.cartErr(res.t > p.traj.T/2)));
fprintf(' coppie massime |tau_i| [N m]: %s\n', mat2str(max(abs(res.tau), [], 2), 4));
end

```

A.17 plot_robot_4R.m

```

function h = plot_robot_4R(ax, theta, L, h, frameScale)
% PLOT_ROBOT_4R Disegna il robot 4R e le terne DH di ogni giunto.
%
% h = plot_robot_4R(ax, theta, L)          primo disegno
% h = plot_robot_4R(ax, theta, L, h)      aggiorna un disegno esistente
% h = plot_robot_4R(ax, theta, L, h, frameScale) lunghezza assi terne [m]
%
% Le terne DH 0..4 sono disegnate con asse x in rosso e asse y in verde;
% l'asse z (uscente dal piano) e' indicato dal pallino sull'origine.
% Restituisce la struttura di handle grafici (per animazioni veloci).
if nargin < 4, h = []; end
if nargin < 5 || isempty(frameScale), frameScale = 0.15*sum(L)/2.5; end
[~, ~, ~, ~, Tall] = fk_4R(theta, L);
P = squeeze(Tall(1:2, 4, :)); % 2x5: origini delle terne 0..4
xa = zeros(2,5); ya = zeros(2,5);
for i = 1:5
    xa(:,i) = P(:,i) + frameScale*Tall(1:2,1,i);
    ya(:,i) = P(:,i) + frameScale*Tall(1:2,2,i);
end
if isempty(h)
    hold(ax, 'on');
    h.links = plot(ax, P(1,:), P(2,:), '-', 'Color', [0.15 0.25 0.55], 'LineWidth', 5);
    h.joints = plot(ax, P(1,1:4), P(2,1:4), 'o', 'MarkerSize', 9, ...
        'MarkerFaceColor', [1 1 1], 'MarkerEdgeColor', [0.1 0.1 0.1], 'LineWidth', 1.5);
    h.ee = plot(ax, P(1,5), P(2,5), 's', 'MarkerSize', 8, ...
        'MarkerFaceColor', [0.85 0.33 0.1], 'MarkerEdgeColor', 'k');
    for i = 1:5
        h.xax(i) = plot(ax, [P(1,i) xa(1,i)], [P(2,i) xa(2,i)], 'r-', 'LineWidth', 1.5);
        h.yax(i) = plot(ax, [P(1,i) ya(1,i)], [P(2,i) ya(2,i)], 'g-', 'LineWidth', 1.5);
        h.lbl(i) = text(ax, P(1,i) + 0.3*frameScale, P(2,i) - 0.4*frameScale, ...
            sprintf('%d', i-1), 'FontSize', 8, 'Color', [0.3 0.3 0.3]);
    end
    plot(ax, 0, 0, 'k^', 'MarkerSize', 12, 'MarkerFaceColor', [0.6 0.6 0.6]); % base
else
    set(h.links, 'XData', P(1,:), 'YData', P(2,:));
    set(h.joints, 'XData', P(1,1:4), 'YData', P(2,1:4));
    set(h.ee, 'XData', P(1,5), 'YData', P(2,5));
    for i = 1:5
        set(h.xax(i), 'XData', [P(1,i) xa(1,i)], 'YData', [P(2,i) xa(2,i)]);
        set(h.yax(i), 'XData', [P(1,i) ya(1,i)], 'YData', [P(2,i) ya(2,i)]);
        set(h.lbl(i), 'Position', [P(1,i) + 0.3*frameScale, P(2,i) - 0.4*frameScale, 0]);
    end
end
end
end

```

A.18 plot_results_4R.m

```

function plot_results_4R(res, p)
% PLOT_RESULTS_4R Grafici dei risultati: q(t), e(t), tau(t) e percorso cartesiano.
t = res.t;
col = lines(4);

figure('Name', 'q(t): posizioni di giunto', 'Color', 'w');
for i = 1:4

```

```

subplot(2,2,i);
plot(t, rad2deg(res.qd(i,:)), '--', 'Color', [0.4 0.4 0.4], 'LineWidth', 1.5); hold on;
plot(t, rad2deg(res.q(i,:)), '-', 'Color', col(i,:), 'LineWidth', 1.5);
grid on; xlabel('t [s]'); ylabel(sprintf('\theta_%d [deg]', i));
title(sprintf('Giunto %d', i)); legend('riferimento q_d', 'reale q', 'Location', 'best');
end

figure('Name', 'e(t): errore di inseguimento', 'Color', 'w');
subplot(2,1,1);
plot(t, rad2deg(res.e), 'LineWidth', 1.5); grid on;
xlabel('t [s]'); ylabel('e = q_d - q [deg]'); title('Errore nello spazio dei giunti');
legend('e_1', 'e_2', 'e_3', 'e_4', 'Location', 'best');
subplot(2,1,2);
semilogy(t, max(res.cartErr, 1e-12), 'k', 'LineWidth', 1.5); grid on;
xlabel('t [s]'); ylabel('||p_d - p|| [m]'); title('Errore cartesiano dell''end-effector');

figure('Name', 'tau(t): coppie ai giunti', 'Color', 'w');
plot(t, res.tau, 'LineWidth', 1.5); grid on;
xlabel('t [s]'); ylabel('\tau [N m]');
title(sprintf('Coppie ai giunti (controllore: %s)', strrep(p.ctrl.type, '_', ' ')));
legend('\tau_1', '\tau_2', '\tau_3', '\tau_4', 'Location', 'best');

figure('Name', 'Percorso cartesiano', 'Color', 'w');
plot(res.ref.X(1,:), res.ref.X(2,:), 'k--', 'LineWidth', 1.5); hold on;
plot(res.Xee(1,:), res.Xee(2,:), 'r-', 'LineWidth', 1.2);
plot(res.Xee(1,1), res.Xee(2,1), 'ro', 'MarkerFaceColor', 'r');
axis equal; grid on; xlabel('x [m]'); ylabel('y [m]');
legend('traiettoria desiderata', 'end-effector', 'posizione iniziale', 'Location', 'best');
title('Traiettorie nel piano XY');
end

```

A.19 animate_4R.m

```

function animate_4R(res, p)
% ANIMATE_4R Animazione del robot (drawnow) con terne DH e traiettoria target.
%
% Ad ogni frame vengono disegnati: robot, terne DH dei giunti, percorso
% desiderato (tratteggiato), punto target corrente (cerchio verde) e
% scia dell'end-effector (rosso). Se p.viz.saveVideo = true l'animazione
% viene salvata in p.viz.videoFile (solo MATLAB, tramite VideoWriter).
L = p.robot.L;
R = sum(L) * 1.05;
fig = figure('Name', 'Animazione robot 4R', 'Color', 'w');
ax = axes('Parent', fig);
axis(ax, 'equal'); axis(ax, [-R R -R R]); grid(ax, 'on'); hold(ax, 'on');
xlabel(ax, 'x [m]'); ylabel(ax, 'y [m]');
plot(ax, res.ref.X(1,:), res.ref.X(2,:), 'k--', 'LineWidth', 1); % target
hTrace = plot(ax, NaN, NaN, 'r-', 'LineWidth', 1.2); % scia EE
hTarget = plot(ax, NaN, NaN, 'o', 'MarkerSize', 10, 'LineWidth', 2, 'Color', [0 0.6 0]);
h = plot_robot_4R(ax, res.q(:,1), L, [], p.viz.frameScale);
hTitle = title(ax, '');

vw = [];
if p.viz.saveVideo && exist('OCTAVE_VERSION', 'builtin') == 0
    vw = VideoWriter(p.viz.videoFile, 'MPEG-4');
    vw.FrameRate = round(1/(p.sim.dtOut*p.viz.animStep));
    open(vw);
end
idx = [1:p.viz.animStep: numel(res.t), numel(res.t)];
for k = idx
    if ~ishandle(fig), break; end
    h = plot_robot_4R(ax, res.q(:,k), L, h, p.viz.frameScale);
    set(hTrace, 'XData', res.Xee(1,1:k), 'YData', res.Xee(2,1:k));
    set(hTarget, 'XData', res.Xd(1,k), 'YData', res.Xd(2,k));
    set(hTitle, 'String', sprintf('t = %.2f s | errore EE = %.1f mm', ...
        res.t(k), 1e3*res.cartErr(k)));
    drawnow;
    if ~isempty(vw), writeVideo(vw, getframe(fig)); end
    if p.viz.pause > 0, pause(p.viz.pause); end
end
if ~isempty(vw), close(vw); fprintf('Video salvato in %s\n', p.viz.videoFile); end
end

```

A.20 test_4R.m

```
function ok = test_4R(p, runSymbolic)
% TEST_4R Batteria di test numerici del progetto 4R.
%
%   ok = test_4R()           usa params_4R()
%   ok = test_4R(p)         usa i parametri forniti
%   ok = test_4R(p, true)   include il confronto con la derivazione
%                           simbolica (richiede Symbolic Toolbox)
%
%   Ogni test stampa PASS/FAIL e l'errore misurato; ok = true se tutti
%   i test sono superati.
if nargin < 1 || isempty(p), p = params_4R(); end
if nargin < 2, runSymbolic = false; end
L = p.robot.L(:); r = p.robot;
rng(1); % riproducibilita'
nRand = 200;
results = {};

% ----- 1. Cinematica diretta: confronto con formule in forma chiusa
err = 0;
for k = 1:nRand
    q = 2*pi*rand(4,1) - pi;
    [T, x, y, phi] = fk_4R(q, L);
    ph = cumsum(q);
    err = max([err, abs(x - sum(L.*cos(ph))), abs(y - sum(L.*sin(ph))), ...
        abs(angdiff(phi, sum(q))), abs(det(T(1:3,1:3)) - 1)]);
end
results(end+1,:) = {'FK: x, y, phi = forma chiusa', err, 1e-12};

% ----- 2. Cinematica inversa: FK(IK(p)) = p (entrambi i gomiti, psi noto)
err = 0; errQ = 0;
for k = 1:nRand
    q = 2*pi*rand(4,1) - pi;
    [~, x, y, phi] = fk_4R(q, L);
    psi = sum(q(1:3));
    for el = {'up', 'down'}
        [qi, info] = ik_4R(x, y, phi, L, el{1}, psi);
        [~, xi, yi, phii] = fk_4R(qi, L);
        err = max([err, abs(xi-x), abs(yi-y), abs(angdiff(phii, phi))]);
        if ~info.reachable, err = Inf; end
    end
    % una delle due soluzioni deve coincidere con q di partenza
    [~, info] = ik_4R(x, y, phi, L, 'up', psi);
    d = min(max(abs(angdiff(info.solutions(:,1), q)), max(abs(angdiff(info.solutions(:,2), q))));
    errQ = max(errQ, d);
end
results(end+1,:) = {'IK: FK(IK(p)) - p (gomito alto/basso)', err, 1e-9};
results(end+1,:) = {'IK: q originale tra le 2 soluzioni', errQ, 1e-7};

% ----- 3. IK con psi automatico (default)
err = 0;
for k = 1:nRand
    q = 2*pi*rand(4,1) - pi;
    [~, x, y, phi] = fk_4R(q, L);
    qi = ik_4R(x, y, phi, L);
    [~, xi, yi, phii] = fk_4R(qi, L);
    err = max([err, abs(xi-x), abs(yi-y), abs(angdiff(phii, phi))]);
end
results(end+1,:) = {'IK: FK(IK(p)) - p (psi automatico)', err, 1e-9};

% ----- 4. Punto non raggiungibile
[qi, info] = ik_4R(2*sum(L), 0, 0, L);
results(end+1,:) = {'IK: punto fuori dallo spazio di lavoro', double(info.reachable ||
    ~all(isnan(qi))), 0.5};

% ----- 5. Jacobiano e derivata vs differenze finite
err = 0; h = 1e-6;
for k = 1:20
    q = 2*pi*rand(4,1) - pi; dq = 2*rand(4,1) - 1;
    [J, Jdot] = jacobian_4R(q, L, dq);
    Jn = zeros(3,4);
    for j = 1:4
        e = zeros(4,1); e(j) = h;
        [~, xp, yp, pp] = fk_4R(q+e, L); [~, xm, ym, pm] = fk_4R(q-e, L);
        Jn(:,j) = [xp-xm; yp-ym; angdiff(pp, pm)]/(2*h);
    end
    Jdn = (jacobian_4R(q + h*dq, L) - jacobian_4R(q - h*dq, L))/(2*h);
```

```

    err = max([err, max(abs(J(:)-Jn(:))), max(abs(Jdot(:)-Jdn(:)))]);
end
results(end+1,:) = {'Jacobiano e dJ/dt vs differenze finite', err, 1e-6};

% ----- 6. M(q) simmetrica e definita positiva
err = 0; minEig = Inf;
for k = 1:nRand
    q = 2*pi*rand(4,1) - pi;
    M = dynamics_4R(q, zeros(4,1), r);
    err = max(err, max(max(abs(M - M.'))));
    minEig = min(minEig, min(eig(M)));
end
results(end+1,:) = {'M(q) simmetrica', err, 1e-12};
results(end+1,:) = {'M(q) definita positiva (-min autovalore)', -minEig, 0};

% ----- 7. dM/dt - 2C antisimmetrica (proprieta' di passivita')
err = 0;
for k = 1:20
    q = 2*pi*rand(4,1) - pi; dq = 2*rand(4,1) - 1;
    [~, C] = dynamics_4R(q, dq, r);
    Mdot = (dynamics_4R(q + h*dq, dq, r) - dynamics_4R(q - h*dq, dq, r))/(2*h);
    N = Mdot - 2*C;
    err = max(err, max(max(abs(N + N.'))));
end
results(end+1,:) = {'dM/dt - 2C antisimmetrica', err, 1e-6};

% ----- 8. G(q) = gradiente dell'energia potenziale
err = 0;
for k = 1:20
    q = 2*pi*rand(4,1) - pi;
    [~, ~, G] = dynamics_4R(q, zeros(4,1), r);
    Gn = zeros(4,1);
    for j = 1:4
        e = zeros(4,1); e(j) = h;
        [~, Vp] = energy_4R(q+e, zeros(4,1), r); [~, Vm] = energy_4R(q-e, zeros(4,1), r);
        Gn(j) = (Vp - Vm)/(2*h);
    end
    err = max(err, max(abs(G - Gn)));
end
results(end+1,:) = {'G(q) = dV/dq', err, 1e-6};

% ----- 9. Conservazione dell'energia in evoluzione libera (tau = 0)
x0 = [deg2rad([30 -20 40 10]).'; zeros(4,1)];
f = @(t,x) [x(5:8); freeAcc(x, r)];
[~, X] = ode45(f, [0 2], x0, odeset('RelTol', 1e-10, 'AbsTol', 1e-12));
E = zeros(size(X,1),1);
for k = 1:size(X,1)
    [Ek, Ep] = energy_4R(X(k,1:4), X(k,5:8), r); E(k) = Ek + Ep;
end
results(end+1,:) = {'Energia costante con tau = 0 (deriva)', max(abs(E - E(1))), 1e-5};

% ----- 10. Inseguimento con coppia calcolata e modello esatto
pt = p;
pt.ctrl.type = 'computed_torque'; pt.ctrl.modelScale = 1; pt.ctrl.tauMax = Inf;
pt.traj.T = 3; pt.traj.tHold = 0.5; pt.sim.q0Offset = zeros(1,4); pt.sim.dq0 = zeros(1,4);
pt.sim.dtOut = 0.05;
res = simulate_4R(pt);
results(end+1,:) = {'Coppia calcolata: max errore cartesiano', max(res.cartErr), 1e-4};

% ----- 11. (opzionale) confronto con derivazione simbolica
if runSymbolic
    S = derive_dynamics_4R_sym(p);
    results(end+1,:) = {'Simbolico vs dynamics_4R', S.maxErr, 1e-9};
    results(end+1,:) = {'Lagrange diretto vs M ddot q + C dq + G', S.maxResLagrange, 1e-9};
end

% ----- report
fprintf('\n%-46s %12s %10s %s\n', 'TEST', 'ERRORE', 'SOGLIA', 'ESITO');
fprintf('%s\n', repmat('-', 1, 80));
ok = true;
for k = 1:size(results,1)
    pass = results{k,2} <= results{k,3};
    ok = ok && pass;
    if pass, s = 'PASS'; else, s = 'FAIL'; end
    fprintf('%-46s %12.3e %10.1e %s\n', results{k,1}, results{k,2}, results{k,3}, s);
end
fprintf('%s\n', repmat('-', 1, 80));
if ok, fprintf('TUTTI I TEST SUPERATI\n\n'); else, fprintf('ALCUNI TEST NON SUPERATI\n\n'); end
end

```

```
function ddq = freeAcc(x, r)
[M, C, G] = dynamics_4R(x(1:4), x(5:8), r);
ddq = M \ (-C*x(5:8) - G);
end

function d = angdiff(a, b)
d = mod(a - b + pi, 2*pi) - pi;
end
```